

ePACS Offline Installer

Solution Architecture & Design

Industry-Grade Reference Document for Technology, Architecture, and Client Stakeholders

Version: 1.2

Status: Released for Review

Date: May 2026

Owner: NLPSV Engineering — ePACS Installer Team

PROJECT REFERENCES

BRD: PACS_ERP_Offline_Installer_BRD

Plan: ePACS_Offline_Installer_Plan v3.5 (99 gaps, 24 sections, 14 appendices)

Implementation: l3_installer .NET 8 solution v1.0 (with file sync, heartbeat, observability framework)

Table of Contents

Revision History

Version	Date	Author	Summary of Changes
0.1	Mar 2026	Installer Eng	Initial draft from BRD and v1 plan
0.5	Apr 2026	Architecture	Aligned with v3.0 plan; added DDL drift, Percona toolkit, Traceability
0.9	Apr 2026	Architecture	Aligned with v3.1 plan; rural resilience, Traceability integration
1.0	May 2026	Architecture	Aligned with v3.2 plan; added supply chain (SLSA L3), threat model, fleet mgmt, file sync, heartbeat. Maps to v1.0 .NET implementation.
1.1	May 2026	Architecture	Aligned with v3.4 plan; added Section 5.7 Observability framework (10 NuGet packages), Section 6.6 deletion / amendment / financial-data sync (Section 23 of plan), expanded scenarios with G93–G99, new diagrams 17–20, updated glossary.
1.2	May 2026	Architecture	Aligned with v3.5 plan; added Section 6.7 AUTO_INCREMENT seed and BIGINT ID-space partitioning (Section 24 of plan); new diagram 21 (ID space allocation, sequence infra, overflow monitoring); updated glossary, scenarios, implementation status.

Approvals

Role	Name	Signature	Date
Engineering Lead			
Architecture Lead			
Security Lead			
DB Lead			
DevOps Lead			
Sync / API Lead			
Product Owner (NLPSV)			
State Federation IT Rep			
CxO Delegate			

1. Executive Summary

ePACS is a cooperative-banking ERP deployed at thousands of Primary Agricultural Credit Societies (PACS) across India. Each PACS operates as an offline-first node — daily business must continue regardless of internet connectivity, while still reconciling with the National Level Data Repository (NLDR) and being subject to centralized governance from NLPSV (National-Level PACS Software Vendor) and CoopsIndia HQ.

This Solution Architecture & Design document describes a signed, governed, resilient Windows installer and accompanying runtime services that deliver the entire ePACS stack — MySQL 8.4 LTS, Microsoft Garnet (cache), Apache Kafka 3.7 LTS (KRaft, single-node), Kestrel-hosted web and business services, a Sync Agent for bidirectional NLDR synchronization, an Installer Agent for ongoing operational automation, and the Traceability module for compliance-grade audit. It is the reference for stakeholders across Architecture, Engineering, Security, DB, DevOps, Operations, and the Client.

1.1 What this document covers

- The end-to-end solution architecture across PACS node and NLDR central.
- Lifecycle modes: Install, Upgrade, Repair, Backup, Restore, Uninstall, Hotfix, Self-Update.
- The .NET 8 solution layout, project orthogonality, traceability, observability, and security posture.
- How data consistency, coherence, and integrity are maintained across processes, services, and node lifecycle.
- How online ↔ offline transitions are handled with no data loss, no duplicates, and bounded backlog.
- Patterns and best practices: transactional outbox, expand-migrate-contract, side-by-side junction-flip upgrade, schema fingerprinting, SLSA L3 build provenance, ATA kill-switch.
- Key scenarios covered: power-cut mid-upgrade, 30-day offline, USB media corruption, AV interference, GPO override, schema drift, key compromise, large-dataset migration.

1.2 Key design decisions at a glance

Area	Decision
Installer tooling	WiX v4 Burn + C# Managed Bootstrapper Application (.NET 8 self-contained, win-x64)
Web host	Kestrel under Windows Service — no IIS dependency
Database	MySQL 8.4 LTS Community (ZIP distribution) — installer-owned my.ini
Cache	Microsoft Garnet (OSS, .NET 8) under ICache abstraction (Redis OSS fallback ready)
Eventing	Apache Kafka 3.7 LTS, KRaft mode, single-node, with Temurin JRE 17 LTS
Schema migrations	DbUp (.NET native) + Percona pt-online-schema-change for COPY-algorithm DDL on > 1M rows
Physical backup	Percona XtraBackup 8.4 (incremental + full); mysqlsh for portability
Sync transport	ISyncTransport abstraction (HTTP/mTLS default, Disabled mode for pilot)
Signing	EV Authenticode + RFC 3161 timestamping; HSM/Key Vault backed; SLSA L3

Area	Decision
Provenance	in-toto attestations; reproducible builds verified across two VM images
Compromise response	ATA (Anti-Tamper Allowlist) signed by Release CA; refreshed via sync
Backup encryption	AES-256-GCM with certificate-wrapped per-backup symmetric key
Audit	Hash-chained audit log + Traceability module (11 partitioned tables, geo-tagged)
Resilience	Power-cut recoverable state machine; bounded outbox; chunked sync resume
Localization	en-US, en-IN, hi-IN, mr-IN, te-IN, ta-IN with Indic test corpus and locale-safe parsing analyzer

1.3 Outcomes

- Field operators install or upgrade ePACS in under 15 minutes from a single signed USB package.
- Business operations continue across power loss, NLDR outages, and intermittent 4G connectivity.
- All data corrections, upgrades, and config changes are auditable, signed, and tamper-evident.
- HQ has fleet-wide visibility (opt-in) without exposing PII or business data.
- Compliance posture: SLSA L3, STRIDE-modeled, multi-locale, key-escrowed, and DR-rehearsed quarterly.

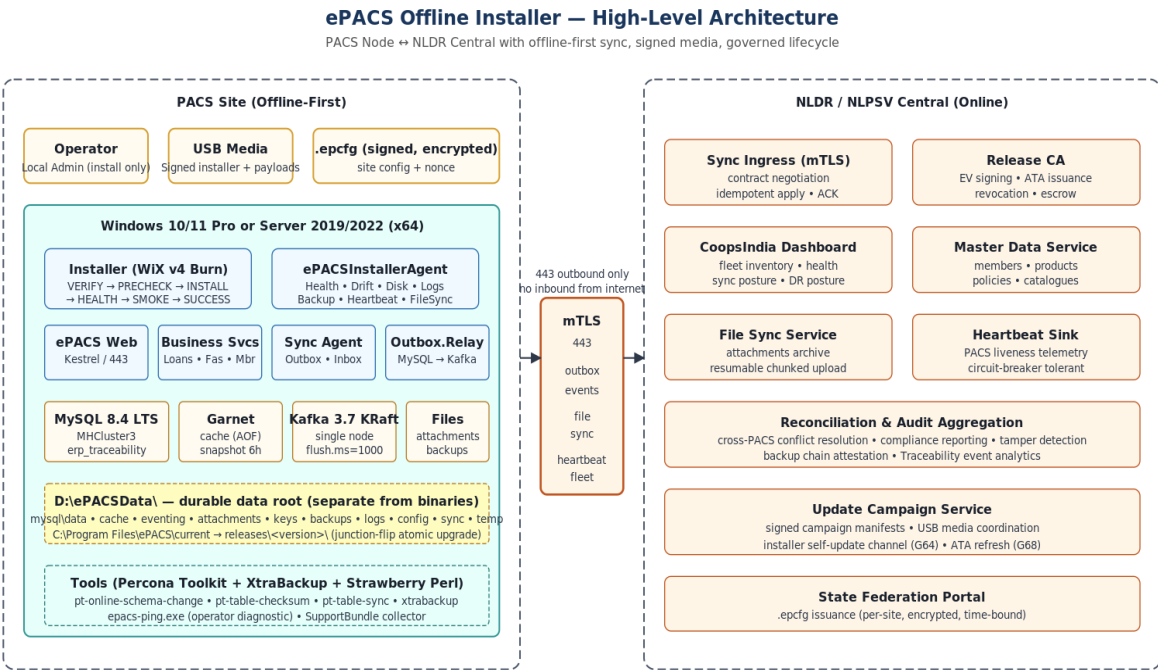


Figure: ePACS Offline Installer — High-Level Architecture (PACS site, NLDR central, governed media supply chain)

2. Problem Statement

2.1 Operating context

PACS sites are typically rural cooperative banks that operate one to twenty users, run on Windows 10/11 Pro or Windows Server 2019/2022 hardware ranging from 8 GB RAM small-PACS hosts up to 32 GB RAM DCCB hub machines. Many sites face daily power outages, intermittent 4G connectivity, ambient temperatures of 35–45 °C, monsoon humidity, and operators with limited technical training. UPS coverage is recommended but not guaranteed. Internet connectivity is opportunistic; some sites are effectively air-gapped for extended periods.

2.2 The core challenges

Challenge	Why it matters
Offline-first authority	Local MySQL must remain the source of truth for business; NLDR sync cannot be a critical path for daily operations.
Governed installation	Every install/upgrade affects regulatory data; must be signed, verified, audited, and reversible.
Media-borne distribution	USB courier is the only reliable channel to many sites; installer must verify integrity end-to-end.
Power-cut tolerance	Any operation can be interrupted by power loss; nothing should leave the system in an unrecoverable state.
Schema evolution at scale	1,057 tables, 81 FKs, 4,556 LOB columns, mixed charsets; upgrades must run in minutes-to-hours, not days.
Large datasets	100+ GB datasets at DCCB hubs; logical dump-reload is impractical; physical incremental needed.
Schema drift	Hundreds of nodes accumulate manual DDL fixes; upgrades must detect and refuse breaking drift.
Compliance and audit	Every business change, override, sync conflict, and operator action must be tamper-evidently logged.
Supply-chain integrity	EV signing alone is insufficient; build pipeline itself must be provenance-attested (SLSA L3).
Field operator UX	Operators must succeed without engineering assistance; silent installs and one-click diagnostics.
Localization	Indic scripts (Hindi, Marathi, Telugu, Tamil) must roundtrip without corruption.
Restore-to-new-machine	Hardware failures require restore on different machines — including cross-profile restores.

2.3 What the solution must NOT do

- Block business operations when NLDR is unreachable.
- Lose data on power-cut, AV interference, USB corruption, or operator error.
- Require live internet for installation, upgrade, or routine operations.
- Trust unsigned, tampered, revoked, or unrecognized installer artifacts at any stage.

- Permit destructive actions (purge, force-upgrade) without governance Override Token.
- Expose business data, secrets, or PII through fleet telemetry or support bundles.

2.4 Success criteria (BRD acceptance)

AC	Description
AC-001	Clean offline install from a single signed package on Windows 10 / Server 2019/2022.
AC-002	Installer refuses unsigned or tampered packages.
AC-003	Fresh install creates the data root with correct ACLs.
AC-004	All health endpoints report expected stack and schema versions.
AC-005	Upgrade creates a verified backup before migration.
AC-006	Upgrade failure triggers rollback or locked recovery state with support bundle.
AC-007	Uninstall preserves data by default; purge requires governance token.
AC-008	Restore from encrypted backup succeeds on clean machine with recovery cert.
AC-009	Local transactions continue while NLDR is unreachable.
AC-010	Sync resumes without duplicates after reconnection.
AC-011	DB, cache, and eventing ports are localhost-only.
AC-012	Support bundle contains diagnostics with no plaintext secrets.

3. The Solution Components

The ePACS solution is composed of three concentric tiers — the runtime stack on each PACS node, the installer and operational agent that govern the runtime, and the central NLDR / NLPSV services that anchor governance, sync, and fleet visibility. Each component is interface-driven for substitutability, fault-isolated, and individually testable.

Per-PACS Deployment Topology — Services, Ports, Paths
Localhost-only data plane • dependency-ordered start • atomic junction upgrade

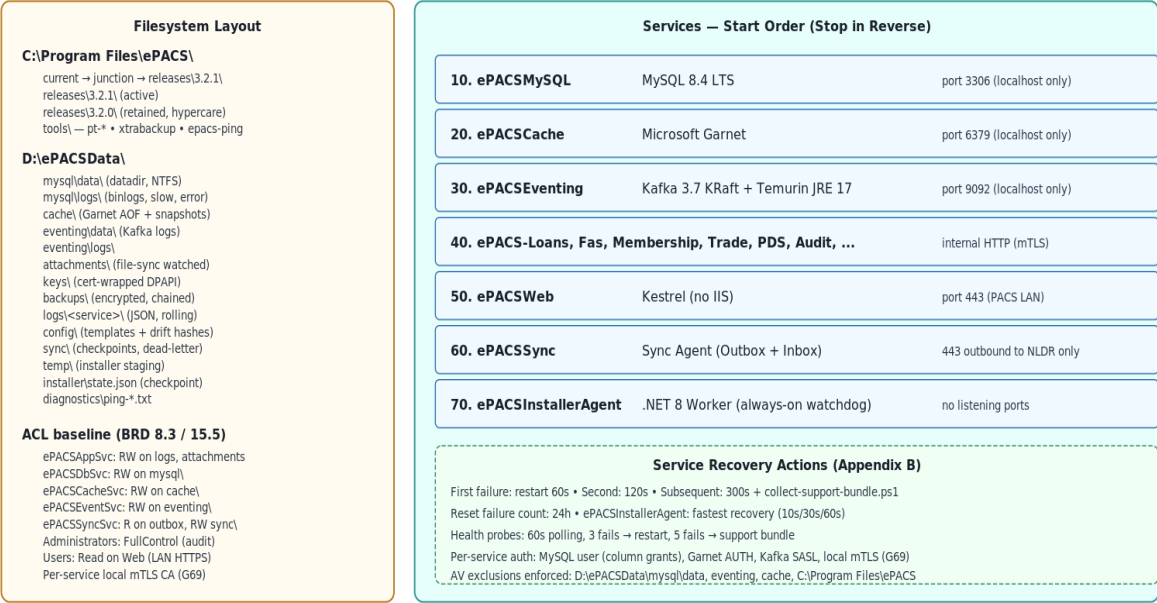


Figure: Per-PACS Deployment Topology — Services, Ports, Filesystem Layout, ACL baseline

3.1 Runtime services on the PACS node

Service	Tech	Purpose
ePACSMySQL	MySQL 8.4 LTS	Single source of truth for business data, audit, sync outbox/inbox checkpoints. Localhost-only port 3306.
ePACSCache	Microsoft Garnet	In-memory cache for session state and lookups. ICache abstraction (Redis fallback). Localhost-only 6379.
ePACSEventing	Kafka 3.7 KRaft	Local event transport for outbox-relay → sync-agent decoupling. KRaft mode (no ZK). Localhost-only 9092.
ePACSWeb	Kestrel	Operator and member UI; HTTPS on PACS LAN. No IIS. Health and sync-conflict pages.
Business Services	ASP.NET Core 8	Loans, FAS, Membership, Trade, PDS, Audit, Asset, HRMS, etc. — internal HTTP with mTLS.
ePACSSync	Sync.Agent	Outbox.Relay (MySQL→Kafka), InboxProcessor (NLDR→PACS), ReconciliationEngine, ConnectivityMonitor.
ePACSInstallerAgent	.NET 8 Worker	Watchdog: Health, Drift, Disk, Logs, Backup orchestration, FileSync,

Service	Tech	Purpose
		Heartbeat, Cert/Time monitors.

3.2 Installer and lifecycle modes

The Installer is a single signed Windows bootstrapper with multiple modes detected by CLI arguments and host state. All modes share VERIFY → PRECHECK common steps. Each mode has explicit rollback semantics.

Mode	Behavior
INSTALL	Fresh install on clean machine. Creates data root, ACLs, services, generates configs from .epcfg, applies firewall, AV exclusions, root-CA bundle, runs first-time DB init, smoke test.
UPGRADE	Side-by-side default. Creates pre-upgrade backup → stages new release → migrates on staging → flips junction → smoke test → on failure: roll back junction or restore backup.
REPAIR	Verifies all payload hashes against manifest; replaces mismatched binaries; regenerates configs from current registry; re-applies ACLs; restarts services. Does not touch data.
BACKUP	Operator- or schedule-triggered. Tier 1/2/3/4 selectable. Encrypted, signed manifest, chain-attested (G87).
RESTORE	Verify package + chain → safety backup of current → stop services → restore staging → validate fingerprint → cutover → smoke. Cross-profile aware (G85).
UNINSTALL	Stops services in reverse order, removes binaries from C:\Program Files\ePACS\, preserves D:\ePACSData\ by default. Purge requires Override Token + typed confirmation.
HOTFIX	Tiny signed package replacing only specified binaries. No schema migration. Stops only affected services. Target downtime < 5 min.
SELF-UPDATE	Installer-only patch (G64). N → N+1 only. Major upgrades remain USB-delivered.

3.3 Cross-cutting components

Component	Role
ManifestVerifier	Parse, hash-verify, and signature-verify release-manifest.yaml; ATA cross-check; SBOM cross-check at install time.
BackupRestore	Tiered backup engine, restore engine, XtraBackup integration, chain attestation (G87), adaptive retention (G84).
SupportBundle	Diagnostic collector with secret redaction, network probes, last-N sync attempts, system snapshot.
SharedKernel	Contracts, Configuration options, Security primitives (ICodeSigner, IAclEngine, ISecretStore, IAuditChain, IFirewallManager), ErrorHandling factory.
Installer.Diagnostics.Ping	epacs-ping.exe — operator one-shot end-to-end health check (G80).
Traceability module	Compliance-grade audit DB (erp_traceability, 11 tables, ULID PKs, partitioned by month, geo-tagged, anomaly rules).
State Federation Portal (NLPSV-side)	Issues per-site encrypted .epcfg with TTL and nonce.
Release CA (NLPSV-side)	Signs releases, ATA, manifests, recovery certs; dual-custody escrow.

Component	Role
CoopsIndia Dashboard (NLPSV-side)	Receives heartbeats and fleet manifests; HQ visibility into install state, health, sync posture.

3.4 Site Configuration Pack (.epcfg)

The .epcfg is the canonical, signed, encrypted descriptor for a specific PACS site. It is issued by the State Federation Portal per installation, with a 7-day TTL and a single-use nonce, encrypted to the PACS site's public key.

```
{
  "pacs_id": "PACS-MH-3-1234",
  "state_code": "MH",
  "language": "hi-IN",
  "data_root": "D:\\ePACSDData",
  "nldr_endpoint": "https://nldr-mh.coopsindia.gov.in",
  "nldr_client_cert_thumbprint": "<sha1>",
  "backup_targets": ["D:\\ePACSDData\\backups", "E:\\PACS-USB"],
  "backup_schedule": { "daily": "02:00", "weekly_day": "Sun", "weekly_time": "03:00" },
  "site_coordinates": { "latitude": 16.5062, "longitude": 80.6480, "accuracy_meters": 50 },
  "anomaly_overrides": { "AN-01": "Disabled", "AN-05": { "start": "07:00", "end": "20:00" } },
  "fleet_manifest_optin": false,
  "nonce": "...",
  "issued_at": "2026-04-29T10:00:00Z",
  "expires_at": "2026-05-06T10:00:00Z"
}
```

4. Architecture and Design

4.1 Architectural style

The architecture is best characterized as a governed, signed, offline-capable distributed system with a strong transactional core. Specifically:

- Hexagonal at the service level — domain logic isolated behind ports (ISyncTransport, ICache, ICodeSigner, IFileSyncTransport, IAuditChain).
- Outbox pattern for sync — MySQL is the system of record; Kafka is transport; NLDR is destination.
- Backend for Frontend (BFF) for ePACS Web — Kestrel surface composes business services.
- Workers and watchdogs — Sync.Agent, Outbox.Relay, Installer.Agent run as Windows services.
- Side-by-side immutable releases — junction-flip enables atomic upgrade and reversible rollback.
- Defense-in-depth security — seven layers from physical to operational.

4.2 Installer state machine

The installer state machine is the operational backbone. Every transition writes a durable checkpoint to D:\ePACSDData\installer\state.json before proceeding. On restart after a power-cut, the installer reads state.json and enters RECOVERY mode, resuming from the last committed phase. This invariant is what makes the installer safe to interrupt at any point.

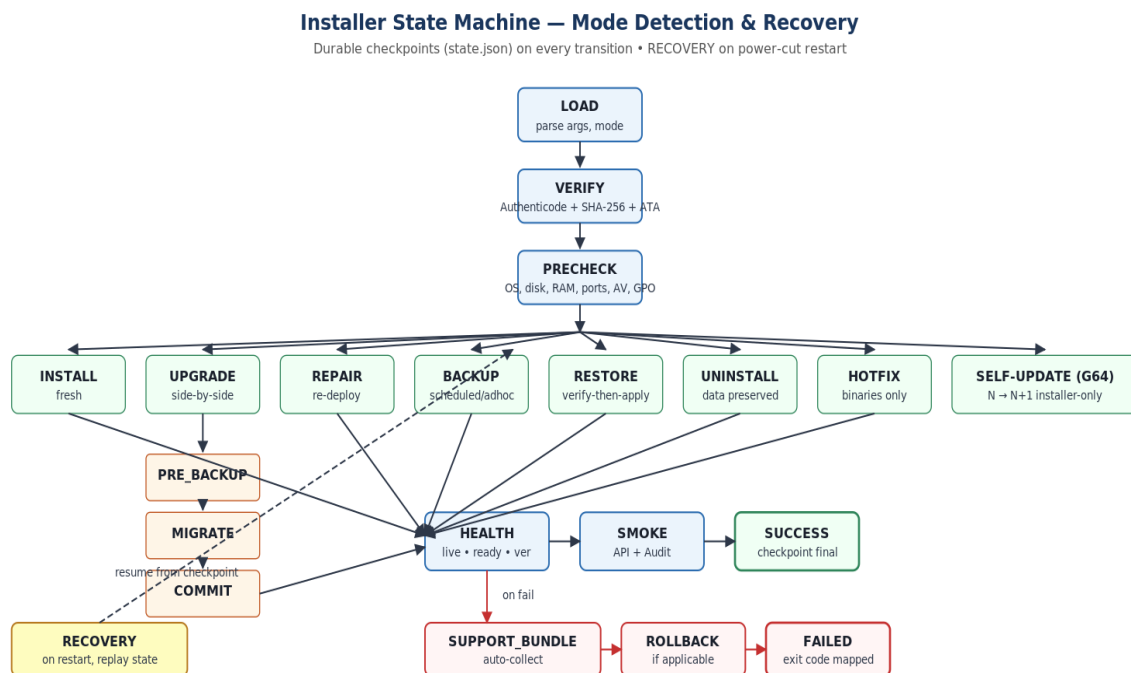
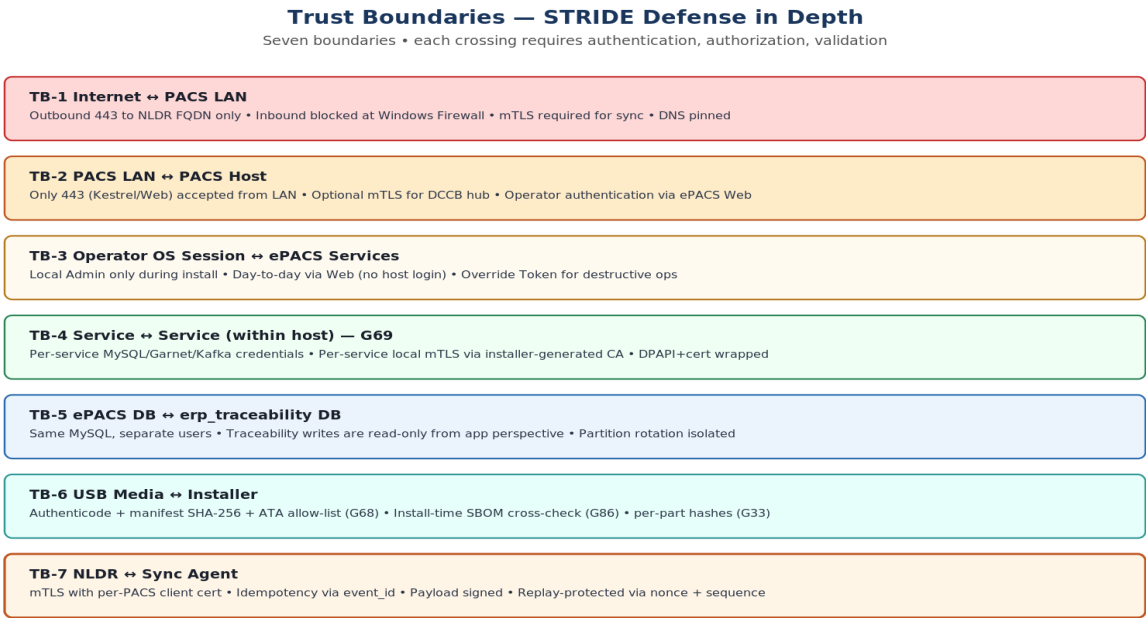


Figure: Installer State Machine — Mode Detection, Common Path, and Recovery

4.3 Per-service authentication and trust boundaries

Localhost-only ports protect the data plane from network attack, but localhost alone cannot prevent service-to-service abuse if any one service is compromised. Defense-in-depth (G69) adds:

- Per-service MySQL users with column-level grants — root is removed from network usage.
- Garnet AUTH password per service principal, rotated at install.
- Kafka SASL/PLAIN with per-service principals.
- Local mTLS via an installer-generated per-PACS Certificate Authority — every service has its own client cert.
- DPAPI + cert-wrapped credential cache — credentials never sit in plaintext on disk.



Defense-in-Depth Layers
Physical → OS Hardening → Network → Service Auth (G69) → Application → Data → Operational

Figure: Seven Trust Boundaries — STRIDE Defense in Depth

4.4 Defense in depth — seven layers

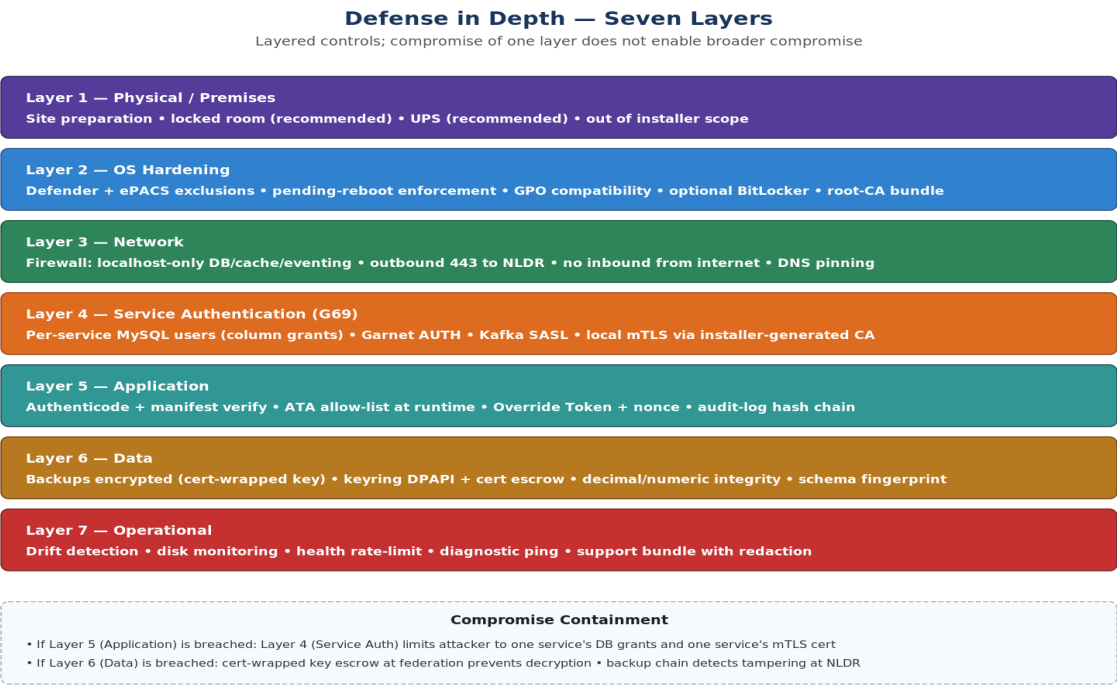


Figure: Defense in Depth — Layered Controls

4.5 Side-by-side upgrade with junction flip

All upgrades — major, minor, and patch — use the side-by-side pattern. New binaries are staged in C:\Program Files\ePACS\releases\<new-version>. The MySQL datadir is copied to staging and migrated there. Only after staging passes smoke-test do we flip the C:\Program Files\ePACS\current junction atomically. If anything fails before the flip, the previous version is untouched. If anything fails after the flip, we flip back.

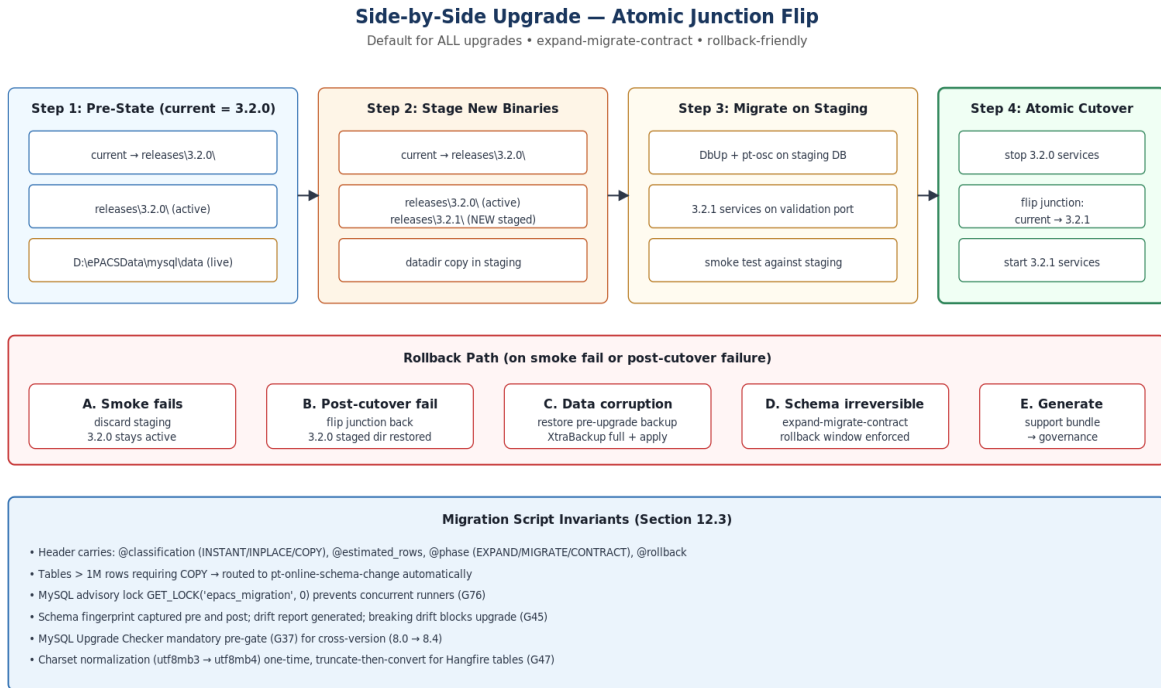


Figure: Side-by-Side Upgrade — Atomic Junction Flip with Rollback Paths

4.6 Schema migration architecture

ePACS has a 1,057-table schema with 81 foreign keys, 4,556 LOB columns, mixed charsets (utf8mb3 in 24 Hangfire tables, utf8mb4 in 1,033 business tables), and AUTO_INCREMENT values approaching 24 % of BIGINT max for one key table. Migration is a first-class architectural concern.

- DbUp runs forward-only versioned scripts (V###__*.sql) with checkpoints in schema_version_registry.
- Each script is classified INSTANT / INPLACE / COPY by its DDL header; tables > 1M rows requiring COPY route to pt-online-schema-change automatically.
- Schema fingerprinting captures INFORMATION_SCHEMA snapshots before and after each migration; drift severity (Benign / Compatible / Breaking) gates the upgrade.
- MySQL Upgrade Checker is a mandatory pre-gate for cross-version transitions (8.0 → 8.4).
- Expand-migrate-contract pattern is enforced — irreversible DDL is split across releases with an explicit rollback window.
- MySQL advisory lock GET_LOCK('epacs_migration', 0) prevents concurrent runners (G76).

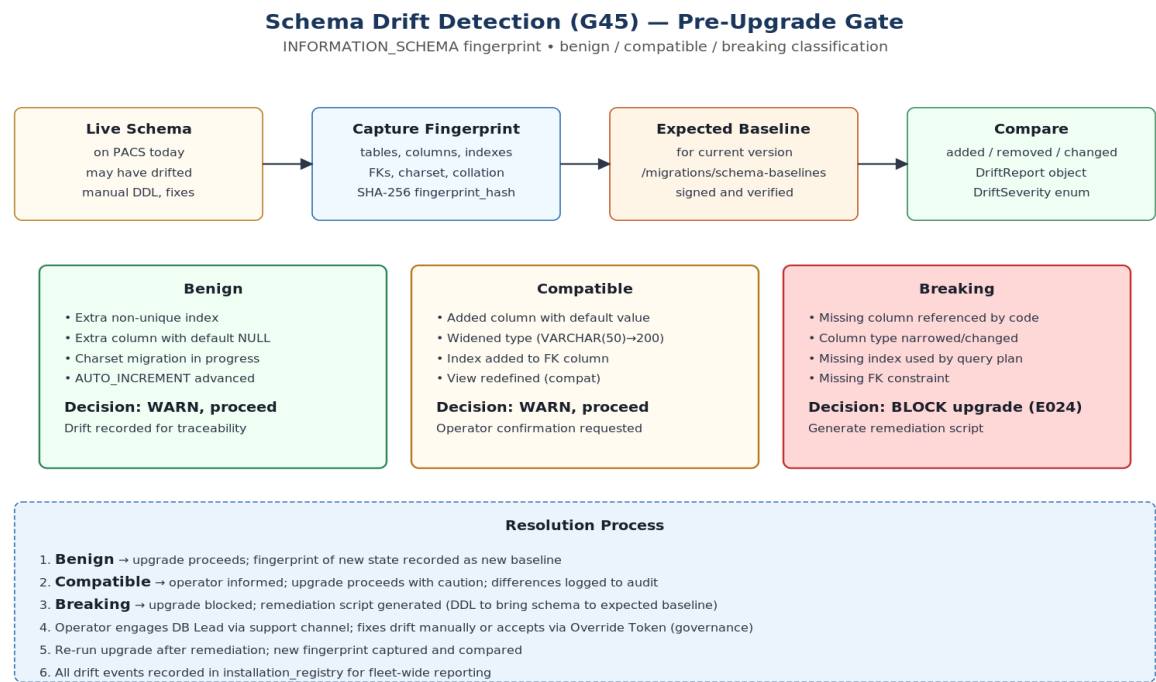


Figure: Schema Drift Detection (G45) — Pre-Upgrade Gate

4.7 Backup and restore architecture

Backup is tiered to balance speed, size, portability, and disaster recovery. Daily incremental physical backups via Percona XtraBackup capture only changed InnoDB pages and complete in minutes even on 100 GB datasets. Weekly full physical backups anchor the chain. Pre-upgrade and monthly logical backups via mysqlsh provide cross-version portability for restore-to-new-machine and restore-to-different-MySQL-version.

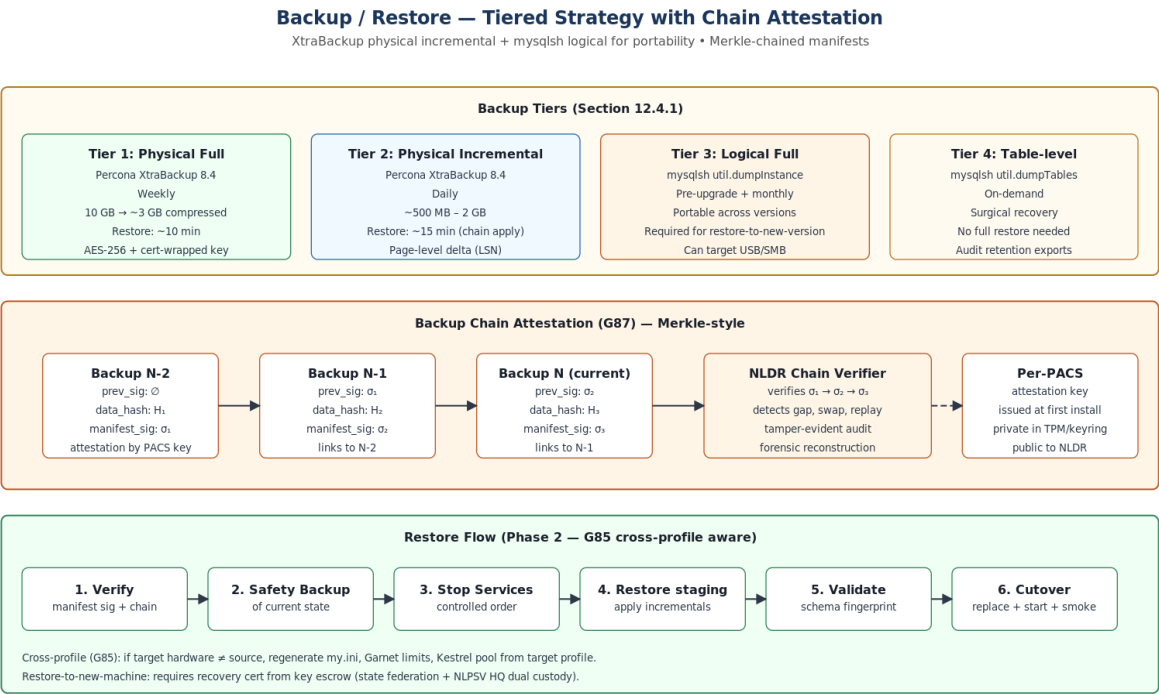


Figure: Backup / Restore — Tiered Strategy with Chain Attestation

4.8 Sync architecture

The Sync Agent is the only component with outbound network access from a PACS site. It implements ISyncTransport with three async methods — SendBatchAsync, ReceiveAsync, ProbeConnectivityAsync — and a circuit breaker that moves the agent through CONNECTED → HALF-OPEN → OPEN states. In OPEN, business operations are entirely unaffected; events queue in MySQL outbox, and the agent probes every 60 seconds.

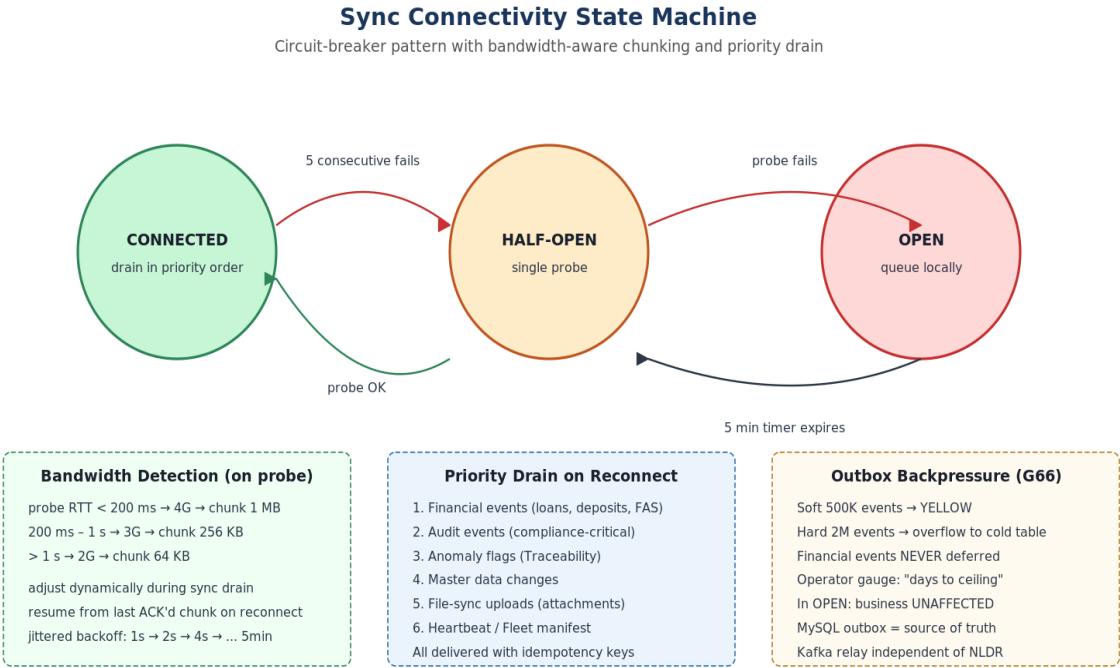


Figure: Sync Connectivity State Machine — Circuit Breaker with Bandwidth Adaptation

4.9 Build provenance and supply chain

Every release artifact is bound to a SLSA L3-style provenance trail: hermetic build VMs, two-person sign-off, in-toto attestations, and reproducible-build verification across two independent build images. The Anti-Tamper Allowlist (ATA) — a small signed file refreshed via sync — provides a kill-switch in case a signing key is compromised.

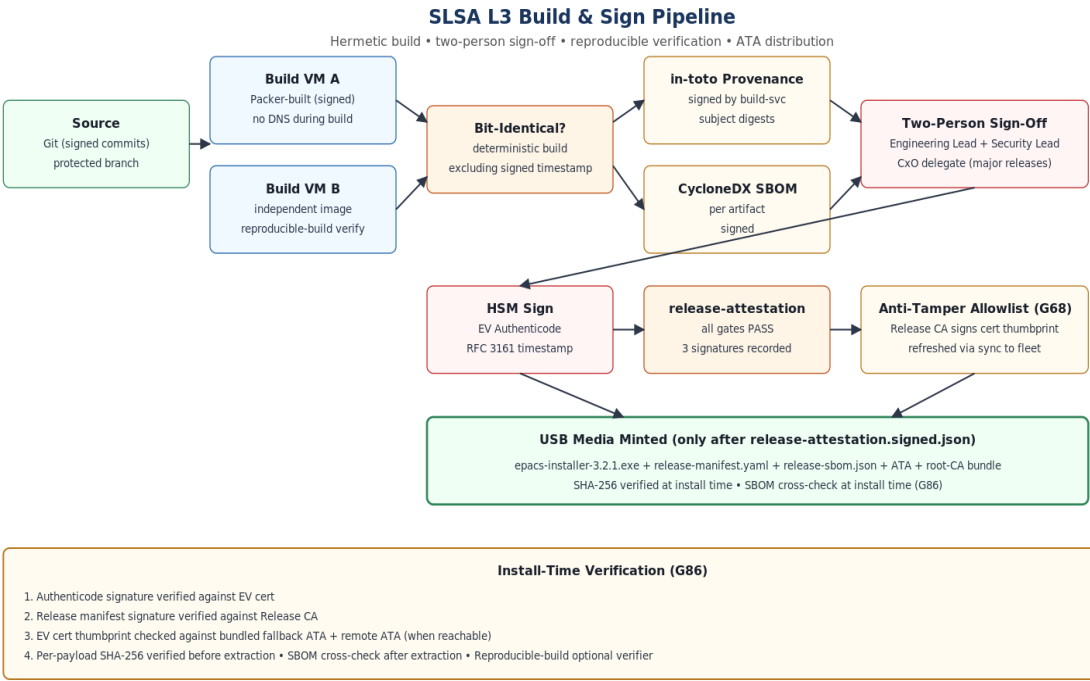


Figure: SLSA L3 Build & Sign Pipeline with ATA Distribution

5. .NET Core Solution Layout — Orthogonality, Traceability, Observability, Security

5.1 Project structure

The solution is decomposed into small, interface-driven, single-responsibility projects. Compile-time references are tightly controlled — concrete projects depend on Abstractions and SharedKernel; never on each other directly. Tests reference the project under test and SharedKernel.

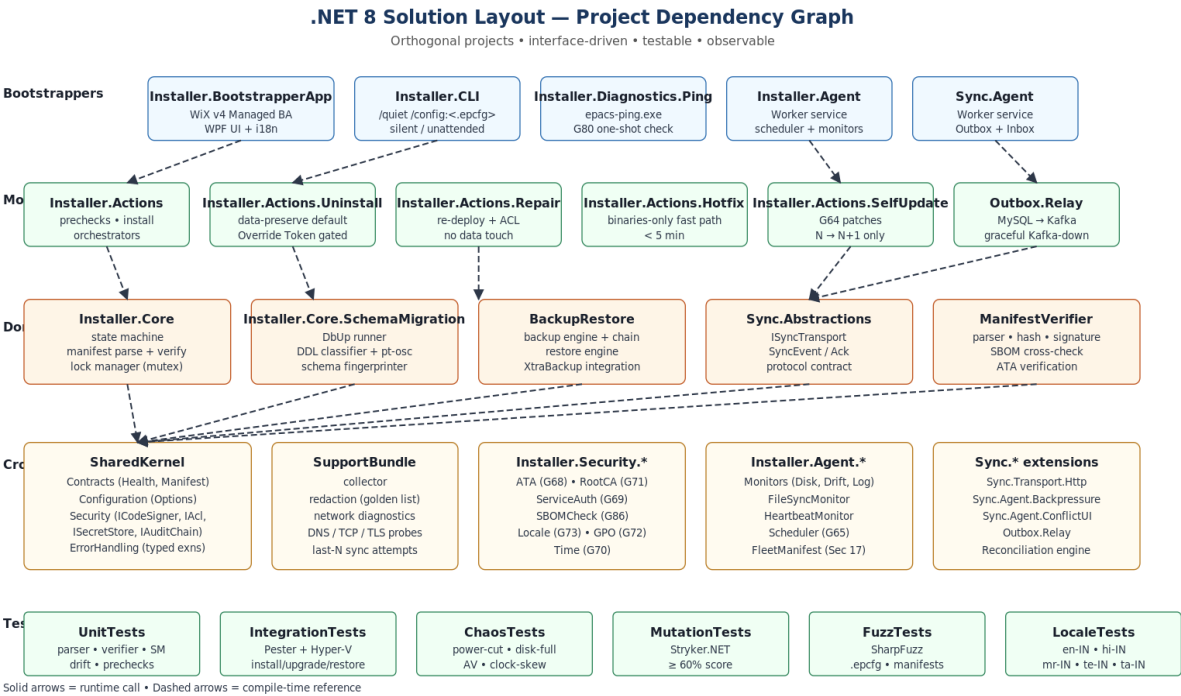


Figure: .NET 8 Solution Dependency Graph — Bootstrappers, Workers, Cores, Cross-Cutting, Tests

5.2 Implemented projects (v1.0)

Project	Public Surface (interfaces / classes)
Installer.CLI	Program.cs entry point — /quiet /config:<.epcfg> mode; exit code mapping
Installer.Core	IInstallerStateMachine, ModeDetector, InstallerLock; ISchemaFingerprinter; IUpgradeEngine
Installer.Actions	IPrecheck (AdminRights, OsVersion, DiskSpace, Ram, Port, PendingReboot); IServiceOrchestrator; IPayloadExtractor; IConfigGenerator; IBinaryDeployer; IDataRootInitializer; UninstallAction
Installer.Agent	Worker (BackgroundService) hosting modular monitors (IMonitor pattern), FileSync, Heartbeat
Installer.Agent / Monitors	DiskSpaceMonitor, ConfigDriftMonitor, LogRotationMonitor (all IMonitor)
Installer.Agent / FileSync	FileSyncMonitor, IFileSyncTransport — bidirectional file sync with hash-based dedup, peak-hour deferral, chunked resume

Project	Public Surface (interfaces / classes)
Installer.Agent / Heartbeat	HeartbeatMonitor, HeartbeatPayload — circuit-breaker-protected fire-and-forget telemetry
Sync.Abstractions	ISyncTransport, SyncEvent, SyncBatchResult, EventAck — transport contract
Sync.Agent	SyncAgentWorker, OutboxRelay (SELECT FOR UPDATE SKIP LOCKED), InboxProcessor, ConnectivityMonitor (Connected/Open/HalfOpen + bandwidth detection), ReconciliationEngine
BackupRestore	IBackupEngine, BackupEngine, IRestoreEngine, BackupManifest
ManifestVerifier	IManifestParser, ManifestParser; IHashVerifier, HashVerifier; ISignatureVerifier, SignatureVerifier; ManifestVerificationService
SupportBundle	ISupportBundleCollector, SupportBundleCollector
SharedKernel	Contracts (ReleaseManifest, SiteConfigPack, InstallationState, ServiceMapEntry, HealthCheckResult); Configuration option classes; Security (IAclEngine, IAuditChain, ICodeSigner, IFirewallManager, ISecretStore); ErrorHandling (IInstallerErrorFactory, InstallerException)

5.3 Orthogonality (separation of concerns)

Each project has one job. State machine knows transitions, not how to talk to MySQL. The schema fingerprinter knows INFORMATION_SCHEMA, not how to render WPF. The Sync Agent knows the outbox protocol, not how to read .epcfg. Cross-cutting concerns — configuration, errors, security — live in SharedKernel and are consumed by interface.

Why this matters: a single fault is contained to one project. A regression in BackupEngine cannot break the state machine. A new sync transport (HTTP, gRPC, future SFTP) plugs in behind ISyncTransport without touching any business code. A different cache (Garnet, Redis OSS, KeyDB) plugs in behind ICache.

5.4 Traceability

Every operation produces a correlation_id (UUID v7) that is propagated end-to-end. This identifier appears in log entries, audit-chain records, support bundles, sync envelopes, and operator-facing error messages. The same ID can be searched across logs and the Traceability database for full reconstruction of any operation.

Layer	Trace artifact
Installer state machine	InstallationState row in installation_registry; correlation_id stamped on every checkpoint
Logs	Structured JSON with correlation_id; daily rolling files under D:\ePACSDData\logs\<service>\
Audit chain	Hash-chained audit_log table linking installer, upgrade, backup, restore, override events
Traceability module	AuditActivity table with ULID PK, geo-tag, OccurredAtUtc, ActorPrincipalId, EventType, ResourceRef, ChangeSetRef
Sync envelope	Each SyncEvent carries event_id (idempotency), idempotency_key, payload_hash, sequence_number, correlation_id
Support bundle	Includes correlation_id index → log file ranges → audit timeline → service status

Layer	Trace artifact
Health endpoint	/health/version returns stack_version, schema_version, sync_protocol_version, manifest_id, build_provenance_id
Backup manifest	backup-manifest.yaml carries manifest_id, prev_signature, signed by per-PACS attestation key (chain attestation)

5.5 Observability

- Structured JSON logs (BRD 17.3) with correlation_id, level, service, host, schema_version.
- Health endpoints (BRD 17.2) on every service: /health/live, /health/ready, /health/version. Caches added (G79) — /ready 10s, /live 1s.
- Local Health Dashboard in ePACS Web — service status, disk usage, last backup, sync lag, cert expiry, drift status, attachment quota.
- Fleet Health Manifest (Section 17 of plan) — privacy-minimized telemetry to CoopsIndia Dashboard, opt-in per .epcfg.
- epacs-ping.exe — single command end-to-end health summary in operator's language, also saved to diagnostics folder.
- Support bundle — secret-redacted, signed, includes DNS / TCP / TLS probes to NLDR plus last 10 sync attempts.
- Mode-aware logging — installer modes log to installer\<mode>-<timestamp>.log; agent monitors log per-monitor.

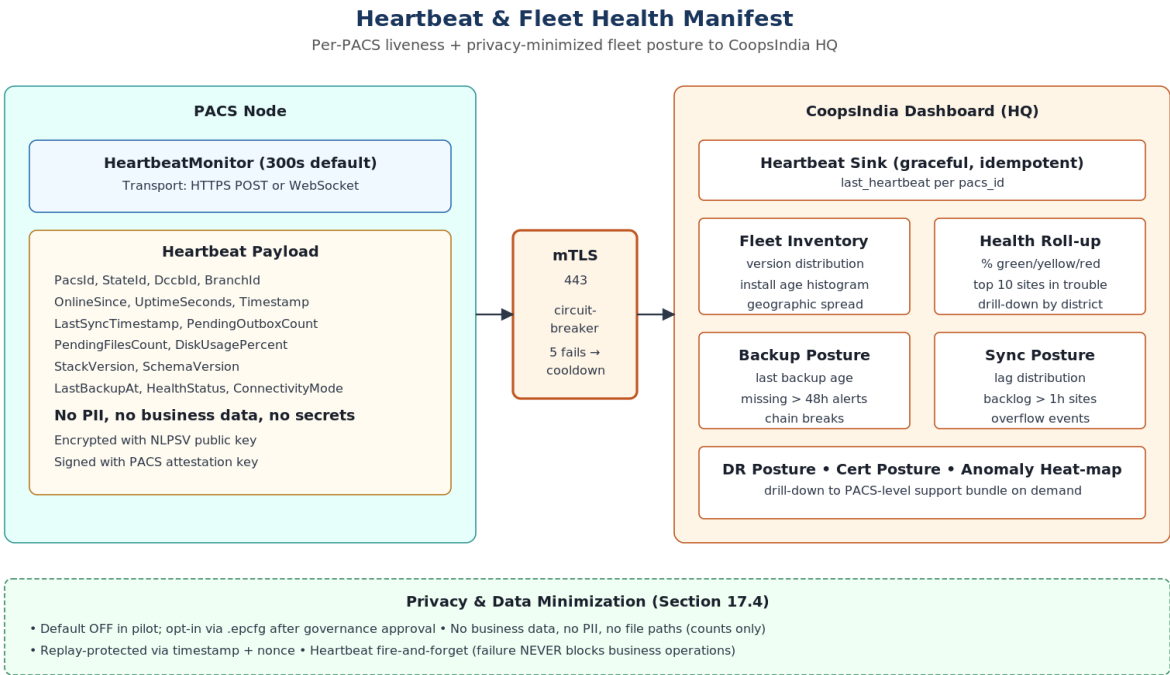


Figure: Heartbeat & Fleet Manifest — Privacy-Minimized HQ Visibility

5.6 Security posture

5.6.1 Identity, secrets, and credentials

- Service principals: ePACSAppSvc, ePACSDbSvc, ePACSCacheSvc, ePACSEventSvc, ePACSSyncSvc — created and ACL'd by installer.
- Credentials encrypted with DPAPI + cert; cached under D:\ePACSData\keys with strict ACLs.
- ISecretStore provides Generate, Store, Retrieve, Rotate, Scan-for-Secrets, ListKeys.
- Key escrow: dual-custody recovery cert at state federation + NLPSV HQ; quarterly key-recovery drill.

5.6.2 Code signing and tamper resistance

- EV Authenticode + RFC 3161 timestamping on every executable, DLL, PowerShell, and manifest.
- ICodeSigner: SignFileAsync (detached CMS/PKCS#7), VerifyFileAsync, ValidateChainAsync.
- Anti-Tamper Allowlist (ATA, G68): signed by Release CA, refreshed via sync, embedded fallback in agent.
- Reproducible builds verified across two build VM images; bit-identical comparison.

5.6.3 Audit chain and tamper detection

- Hash-chained audit log links every install, upgrade, backup, restore, DB-correction, config change, uninstall, override event.
- IAuditChain provides Append, Verify, Anchor, ReadFromCheckpoint.
- Backup chain attestation (G87): each backup manifest carries previous backup's signature → Merkle-style verification at NLDR.
- Configuration drift detection (G22): SHA-256 of every generated config; hourly comparison; drift recorded in support bundle.

5.6.4 Network and access control

- Localhost-only ports for DB, cache, eventing — verified at install and by agent.
- Outbound 443 only to approved NLDR FQDN; mTLS with per-PACS client cert.
- IFirewallManager applies and verifies firewall rules.
- IAclEngine: GenerateRules, ApplyRulesAsync, VerifyAsync — set absolute (SDDL), not delta.

5.6.5 Threat model (STRIDE summary)

Full STRIDE threat model with 28 threats and verifying tests is in the plan's Appendix I. Summary:

Category	Top mitigations
Spoofing	EV signing + ATA allow-list (G68); per-site .epcfg encryption + nonce (G92); JWT Override Token; mTLS to NLDR
Tampering	Authenticode + payload SHA-256; drift detection; backup chain (G87); audit hash chain; schema fingerprint (G45)
Repudiation	Audit log hash-chained; Override Token signed + nonced + reasoned; Traceability captures all DB corrections
Info Disclosure	Support bundle redaction + golden list; encrypted keyring (DPAPI + cert); mTLS sync; .epcfg encryption (G92)

Category	Top mitigations
DoS	Disk monitoring tiers (G24); health endpoint cache + rate-limit (G79); AV exclusions; Garnet AOF quarantine; outbox ceilings (G66)
EoP	Per-service auth + mTLS (G69); nonce replay prevention; HSM/Key Vault + DPAPI; GPO compatibility precheck (G72)

5.7 Observability Framework — Intellect.Erp.Observability + ErrorHandling

All ePACS services — installer agent, sync agent, business services, and the Web frontend — are unified by a standardized observability and error-handling framework consisting of 10 NuGet packages. This framework is the single source of structured JSON logs, typed exceptions, catalog-driven error responses, PII redaction, and audit-hook bridging into the Traceability module. It is what makes a single correlation_id navigable from a user click in ePACS Web all the way to an event landing on the NLDR sync ingress.

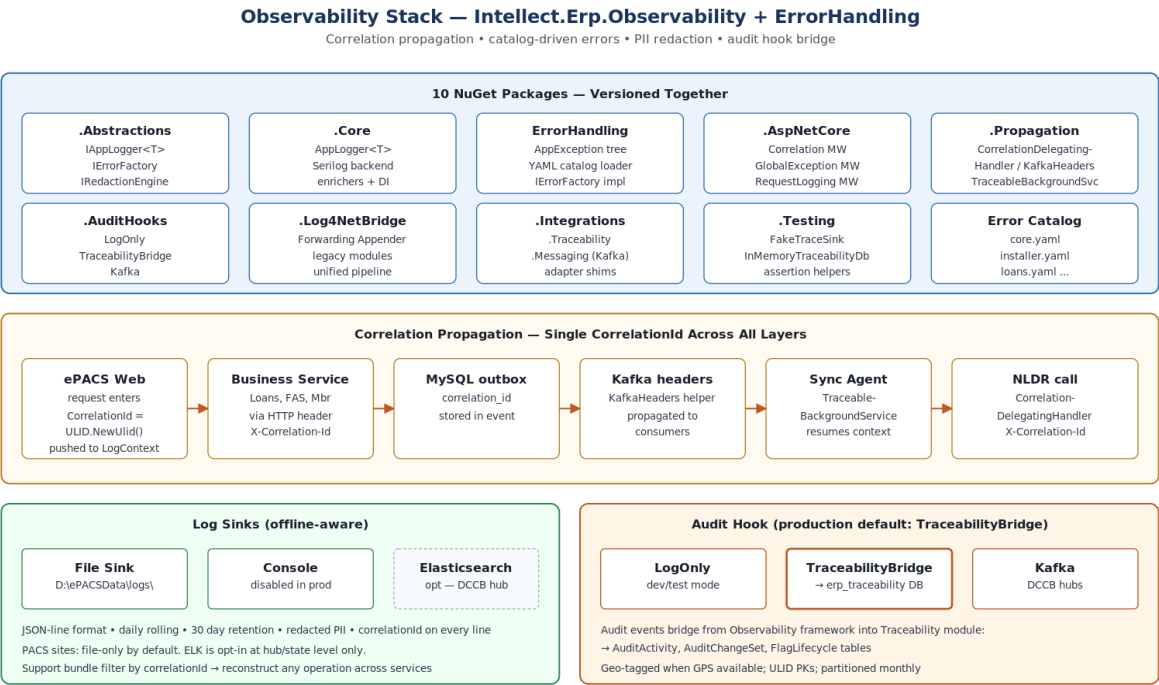


Figure: Observability Stack — 10 NuGet Packages, Correlation Propagation, Audit-Hook Bridge

5.7.1 Package inventory

Package	Role
Intellect.Erp.Observability.Abstractions	Contracts: ILogger<T>, IErrorFactory, IRedactionEngine, IAuditHook, attributes ([Sensitive], [Mask], [DoNotLog])
Intellect.Erp.ErrorHandling	Typed exception hierarchy (AppException), YAML error-catalog loader, IErrorFactory implementation
Intellect.Erp.Observability.Core	AppLogger<T> (Serilog-backed), enrichers, redaction engine, DI extensions
Intellect.Erp.Observability.AspNetCore	Middlewares — Correlation, GlobalException, ContextEnrichment, RequestLogging — for ePACSWeb and business services

Package	Role
Intellect.Erp.Observability.Propagation	CorrelationDelegatingHandler (HTTP), KafkaHeaders helper, TraceableBackgroundService base class
Intellect.Erp.Observability.AuditHooks	LogOnlyAuditHook (dev), TraceabilityBridgeAuditHook (production default), KafkaAuditHook (DCCB hubs)
Intellect.Erp.Observability.Log4NetBridge	SerilogForwardingAppender — captures legacy log4net output through structured pipeline
Intellect.Erp.Observability.Integrations.Traceability	Adapter shim connecting Observability → Traceability module
Intellect.Erp.Observability.Integrations.Messaging	Kafka envelope enricher used by Outbox.Relay and Sync.Agent
Intellect.Erp.Observability.Testing	FakeTraceSink, InMemoryTraceabilityDb, assertion helpers for unit and integration tests

5.7.2 Canonical structured log schema

Every log line — across installer, agents, business services, and Web — conforms to the ELK Field Reference v1 schema. Logs are JSON-line files locally (no ELK dependency at PACS level). Elasticsearch sink is opt-in only at DCCB hub or state level.

```
{
  "@timestamp": "2026-05-04T10:15:30.123Z",
  "level": "Information",
  "messageTemplate": "Upgrade {UpgradeId} migration {ScriptName} completed in {DurationMs}ms",
  "message": "Upgrade UPG-001 migration V025__add_updated_at.sql completed in 1250ms",
  "log.schema": "v1",
  "app": "epacs-installer-agent",
  "env": "Production",
  "machine": "PACS-AP-XYZ-0001",
  "module": "Installer",
  "correlationId": "01J2X5PQ8WZ4A5C2M8YT4F3M6V",
  "tenantId": "AP-XYZ",
  "stateCode": "AP",
  "pacsId": "AP-XYZ-0001",
  "feature": "SchemaUpgrade",
  "operation": "Migrate",
  "checkpoint": "MigrationScriptCompleted",
  "UpgradeId": "UPG-001",
  "ScriptName": "V025__add_updated_at.sql",
  "DurationMs": 1250
}
```

5.7.3 Catalog-driven error responses

Every error the system produces — at installer, agent, service, or Web layer — flows through the IErrorFactory, which maps a stable code (e.g., ERP-INST-PRE-0019) to a YAML catalog entry. The catalog defines the user-facing message, support-facing detail, severity, retry hint, and category. This decouples error presentation from code, supports localization, and makes errors machine-classifiable for analytics.

Code Range	Domain
ERP-INST-PRE-####	Installer prechecks (E001–E099 mapped)

Code Range	Domain
ERP-INST-INS-####	Install actions (service registration, ACL, etc.)
ERP-INST-MIG-####	Schema migration (DbUp failures, drift, charset)
ERP-INST-BAK-####	Backup engine (target, encryption, manifest)
ERP-INST-SYN-####	Sync (NLDR connectivity, idempotency, conflicts)
ERP-INST-HLT-####	Health checks (consecutive failures, degraded state)
ERP-LOA-####	Loans business service
ERP-FAS-####	Financial Accounting business service
ERP-MBR-####	Membership business service

5.7.4 Typed exception hierarchy

```
AppException (abstract, carries code + correlationId + cause)
├── ValidationException      // E001-E099 — Precheck, input validation
├── DataIntegrityException  // schema drift, migration failure, backup corruption
├── DependencyException     // MySQL/Garnet/Kafka unavailable
├── IntegrationException    // NLDR sync failures (retryable=true)
├── ConflictException       // concurrent runner, lock contention
├── SystemException         // unclassified
└── ExternalSystemException // AV interference, GPO, Windows Update conflicts
```

5.7.5 PII redaction in logs and support bundles

The IRedactionEngine ensures sensitive data never appears in log files or support bundles. Redaction rules are declarative (attributes on DTOs) and applied uniformly across all sinks.

Data type	Attribute	Redacted output	Example context
Aadhaar number	[Sensitive(keepLast: 4)]	****-****-1234	Member registration
Mobile number	[Sensitive(keepLast: 4)]	*****5678	Customer details
Account number	[Mask(@"\d{6,}")]	***456789	Loan disbursement
Email	[Sensitive(EmailMask)]	j***@***.com	User profile
Password / secret	[DoNotLog]	(field omitted entirely)	Service credentials
SQL with data	regex pattern	SELECT ... WHERE id=***	Correction tool logs

5.7.6 Audit hook configuration

The IAuditHook bridges any logging event tagged as audit-relevant directly into the Traceability module's AuditActivity / AuditChangeSet / FlagLifecycle tables. Production default at PACS is TraceabilityBridge.

Mode	When	Behavior
LogOnly	Dev / test	Audit events written as structured log entries only
TraceabilityBridge	Production at PACS (default)	Audit events written to erp_traceability DB via ITraceSink with geo-tag, ULID, partition rotation
Kafka	DCCB hubs	Audit events published to Kafka topic for downstream consumers

5.7.7 Correlation propagation across all layers

A single correlationId (ULID) is generated at the originating call (operator click, scheduled job tick, incoming sync command) and propagated through every subsequent operation:

- ePACS Web → CorrelationMiddleware sets it on inbound HTTP.
- Service → service via X-Correlation-Id HTTP header (CorrelationDelegatingHandler).
- Service → MySQL by attaching it to outbox row's correlation_id column.
- MySQL → Kafka via KafkaHeaders helper that copies it onto the message.
- Sync.Agent → resumes context using TraceableBackgroundService base.
- Sync.Agent → NLDR via X-Correlation-Id HTTP header (CorrelationDelegatingHandler).
- Failures → support bundle filename includes the correlationId; reconstruct any operation across services.

5.7.8 Installer Agent as TraceableBackgroundService

The Installer Agent and Sync Agent both extend TraceableBackgroundService. This base class:

- Auto-generates a fresh correlationId per execution cycle.
- Pushes Module/Operation/Feature to Serilog LogContext.
- Catches and logs exceptions with full context; the worker continues operating.
- Records each cycle's outcome to Traceability via AuditHook.

6. Data Consistency, Coherence, and Integrity

Data integrity is the highest-priority non-functional requirement. The system is engineered such that — under normal operation, planned upgrade, accidental power loss, network partition, or hostile tampering — data is either fully consistent and visible, or recoverable to a previous consistent state with audit-evident provenance.

6.1 Local consistency — within a PACS node

6.1.1 ACID and the transactional outbox

Every business write that needs to be synced is performed inside a single MySQL transaction together with the corresponding outbox record. Either both succeed or neither does. This eliminates the classic dual-write problem (business state goes to MySQL, message goes to Kafka, one of them fails leaving inconsistency).

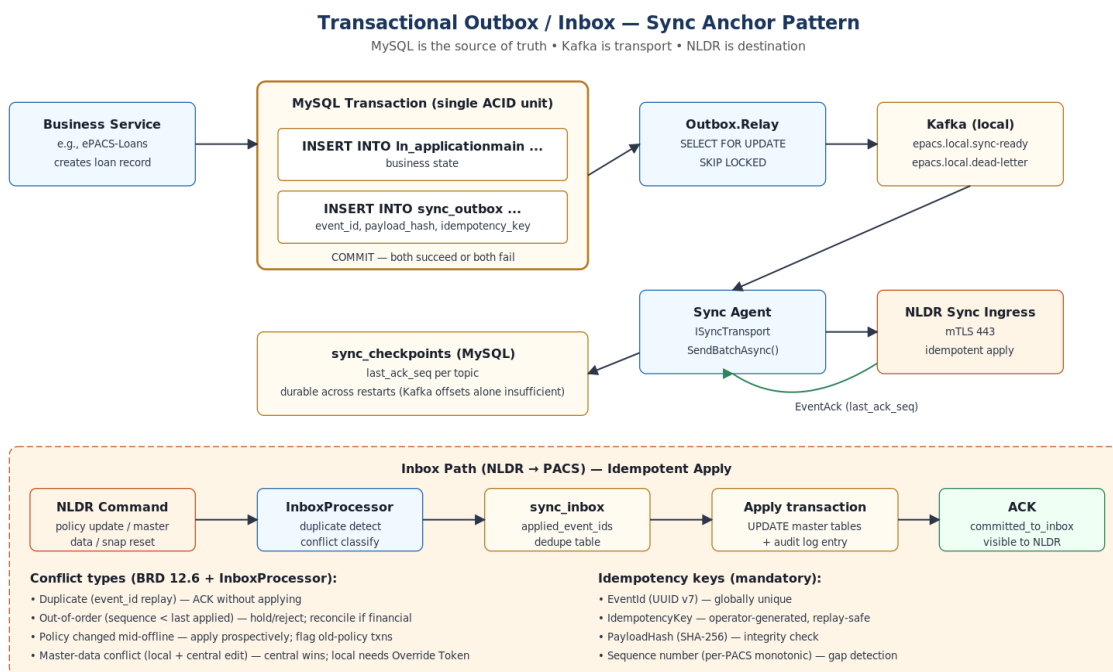


Figure: Transactional Outbox / Inbox — MySQL is the System of Record

6.1.2 InnoDB and Kafka durability settings

```
# my.ini — enforced by installer
innodb_flush_log_at_trx_commit = 1 # fsync redo log on every commit
sync_binlog = 1 # fsync binlog on every commit
innodb_doublewrite = ON # protect against torn pages
innodb_checksum_algorithm = crc32 # detect page corruption
binlog_expire_logs_seconds = 1209600 # 14 days retention (G78)
max_binlog_size = 100M

# kafka.properties
flush.messages = 1
flush.ms = 1000
log.flush.interval.messages = 1
unclean.leader.election.enable = false
```

6.1.3 File-level atomicity

- Every file write uses write-then-rename (atomic on NTFS): write to <file>.tmp, fsync, rename.
- State.json checkpoint writes use the same pattern, ensuring power-cut returns to a valid prior state, never a torn one.
- Junction-flip during upgrade is atomic at the OS level; either current points to old or new, never both or neither.
- Backup manifest writes are signed only after all referenced files are checksummed.

6.1.4 Per-service authentication preserves logical separation

Per-service MySQL grants prevent ePACS-Loans from writing to fas_ledger or audit_log. This is structural — not merely policy — and means a compromise of one service cannot corrupt data owned by another.

6.2 Cross-service coherence

6.2.1 Cache coherence

Garnet is non-authoritative. On every business write, cache entries are invalidated; on cache miss, the value is reloaded from MySQL. If Garnet AOF corrupts (G91), the agent quarantines it and starts with empty cache — no data is lost because MySQL is always the source of truth. Six-hour Garnet snapshots are included in backups for restore optimization, but never trusted authoritatively.

6.2.2 Audit coherence

The Traceability module writes to a separate erp_traceability database on the same MySQL instance. Audit writes are transactional with the originating business write where possible. Where not (e.g., async event recording), the AuditOutbox pattern guarantees eventual consistency — either both eventually appear, or audit alarms fire on the missing event during reconciliation.

6.2.3 Schema coherence (drift detection)

Manual DDL fixes accumulate at field sites over time. The schema fingerprinter snapshots every table, column, index, and FK to detect drift before any upgrade. Breaking drift blocks upgrade entirely; benign drift is recorded but allows progress. Post-upgrade fingerprint becomes the new baseline.

6.3 Convergent consistency — between PACS and NLDR

6.3.1 Idempotency invariants

- EventId — UUID v7, unique per outbox event.
- IdempotencyKey — operator/business-context-derived key for replay safety.
- PayloadHash — SHA-256 over canonicalized JSON; integrity check at receive.
- Sequence number — per-PACS monotonic; gap detection during reconciliation.
- event_id stored in sync_inbox — duplicate detection on receive at PACS.

6.3.2 Conflict resolution

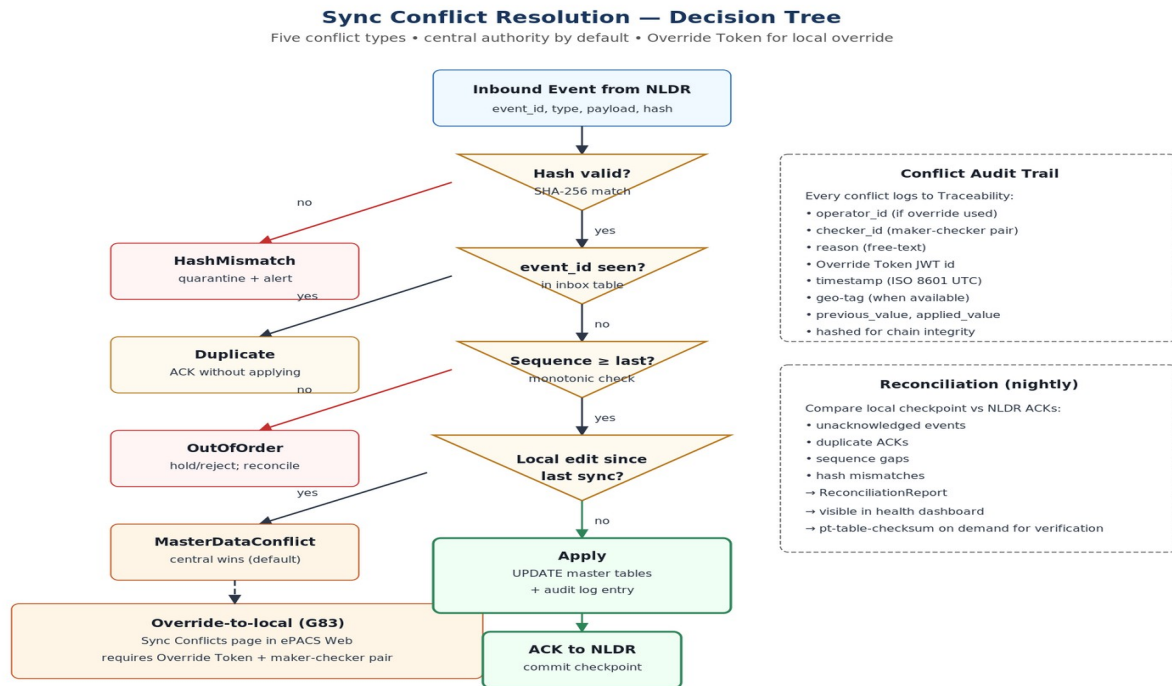


Figure: Sync Conflict Resolution — Decision Tree (5 conflict classes)

6.3.3 Reconciliation

ReconciliationEngine runs nightly and on-demand (post-restore, post-extended-offline). It compares local outbox checkpoints against NLDR ACKs and emits a ReconciliationReport identifying:

- Unacknowledged events — sent but no ACK received.
- Duplicate ACKs — NLDR ACK'd same event_id twice (rare; signals NLDR-side issue).
- Sequence gaps — events missing in the sequence; investigation required.
- Hash mismatches — payload tampering or transport bug.
- Pt-table-checksum drift — row-level inconsistency (e.g., post-restore).

Pt-table-sync is available as an operator-initiated repair tool when drift is detected.

6.4 Tamper-evidence and audit chain

Beyond consistency, the system provides tamper-evidence — silent corruption is detectable. The audit log uses hash chaining: each entry includes the SHA-256 of the previous entry. Any modification to a historical entry breaks the chain and is immediately detectable on next verification. The same pattern protects backup manifests.

6.5 Time monotonicity and clock drift

Audit hash chains and sync ordering depend on monotonic time. The agent monitors clock drift (G16, G70):

- Compare system clock against MySQL NOW() and last-known-good audit timestamp every 30 min.

- Drift > 30 s → log warning.
- Drift > 5 min → block sync; sync re-enabled when drift acceptable.
- Audit-anchor monotonicity check — audit chain must be timestamp-monotonic; violation flagged.
- NTP fallback hierarchy: w32time pool → DCCB hub stratum-2 → optional GPS USB.

6.6 Deletion, Amendment, and Financial-Data Sync

ePACS uses hard DELETES (not soft-delete) for vouchers and certain operational tables, and applies in-place UPDATES for amendments via the Correction Tool. Without explicit treatment, this creates two correctness risks: (a) records synced to NLDR can be deleted locally without NLDR ever knowing, leaving NLDR with stale data; (b) amendments to already-synced data leave NLDR holding old values, with no audit trail of who changed what and why. The architecture addresses both by treating delete and amendment as first-class sync events with full before-state capture, governed by the same outbox pattern used for inserts and updates.

Design principle: "Nothing is ever truly deleted from the sync perspective." Every business mutation that may have already synced to NLDR — INSERT, UPDATE, DELETE, AMENDMENT — produces an outbox event in the same MySQL transaction as the mutation itself. NLDR receives the event and applies the corresponding action (create / update / soft-delete / amendment with audit trail).

6.6.1 Existing local deletion pattern

ePACS already captures pre-deletion state in module-specific tables. There are six such deletion-pattern tables across modules:

Module	Deletion-pattern tables
Financial Accounting (FAS)	fa_voucherdeletionmain, fa_voucherdeletiondetails
Public Distribution (PDS)	pds_purchasedelete, pds_salesdelete
Trading (TR)	tr_purchasedelete, tr_salesdelete
Correction Tool (any module)	CorrectionTool_LoansActivityLog, CorrectionTool_MembershipActivityLog

These tables already capture full row state before the hard DELETE. They are reused as the source for the before_state_json field in the new sync event.

6.6.2 Hard-delete sync flow

Hard-Delete Sync — Capture Before Disposal (Section 23)

"Nothing is truly deleted from the sync perspective" — DELETE events propagate to NLDR for soft-delete

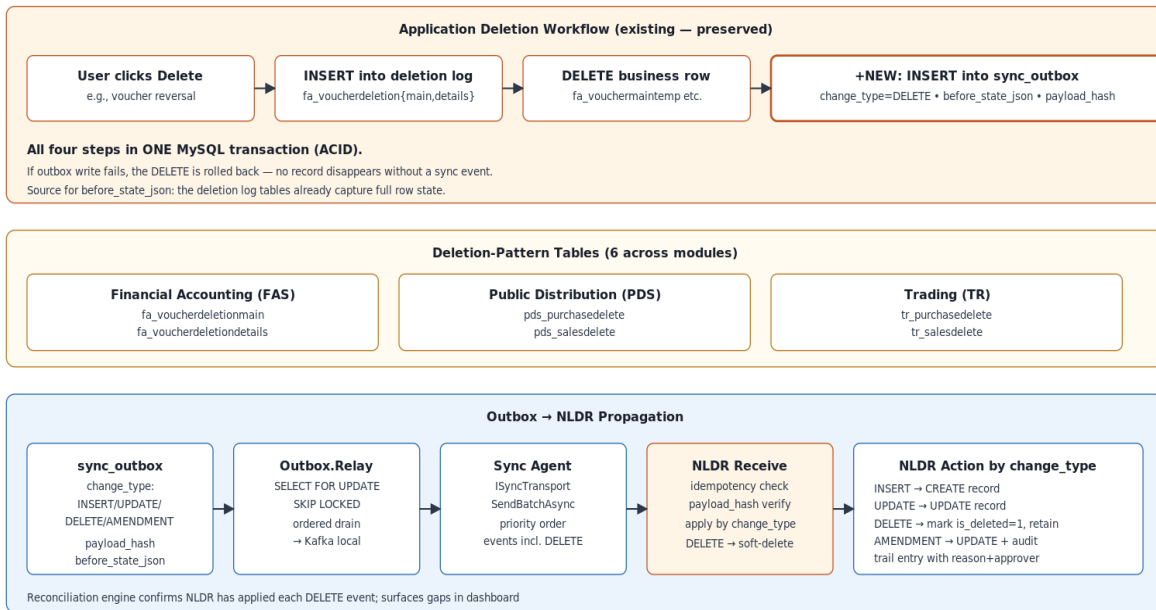


Figure: Hard-Delete Sync — Capture Before Disposal

Every hard DELETE that may have synced to NLDR runs inside one MySQL transaction:

- **Step 1:** INSERT into deletion-pattern table (existing behavior).
- **Step 2:** DELETE from business table (existing behavior).
- **Step 3 (new):** INSERT into sync_outbox with change_type=DELETE, before_state_json captured from the deletion-pattern row, payload_hash = SHA-256(before_state).
- **Step 4:** COMMIT — all-or-nothing. If outbox write fails, the DELETE rolls back. No record disappears without a sync event.

6.6.3 Amendment sync flow

Amendment Sync — Before/After State Capture (Section 23)

In-place UPDATES (corrections) carry full pre/post state to NLDR for governance audit

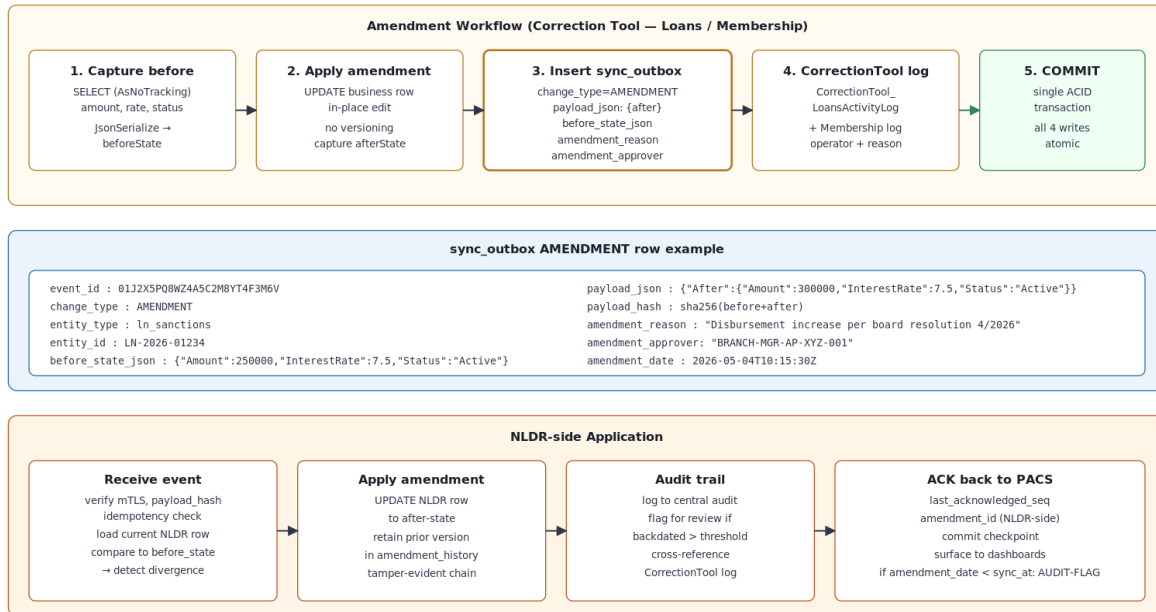


Figure: Amendment Sync — Before/After State Capture

Amendments via the Correction Tool capture both pre- and post-state, plus governance metadata (reason, approver):

```
// Pseudocode — runs inside a single MySQL transaction
var beforeState = SELECT (AsNoTracking) Amount, InterestRate, Status
                  FROM ln_sanctions WHERE Id = @id;

UPDATE ln_sanctions SET Amount=@newAmount, InterestRate=@newRate WHERE Id=@id;

INSERT INTO sync_outbox (
  event_type, change_type, entity_type, entity_id,
  payload_json, before_state_json, payload_hash,
  amendment_reason, amendment_approver, amendment_date,
  correlation_id
) VALUES (
  'DATA_CHANGE', 'AMENDMENT', 'ln_sanctions', @id,
  JSON_OBJECT('After', @afterState),
  JSON_OBJECT(...beforeState...),
  SHA2(CONCAT(beforeJson, afterJson), 256),
  @reason, @approverId, NOW(),
  @correlationId
);

INSERT INTO CorrectionTool_LoansActivityLog (...); -- existing audit
INSERT INTO erp_traceability.AuditActivity (...); -- via AuditHook

COMMIT; -- four writes, atomic
```

6.6.4 sync_outbox schema enhancement

```
ALTER TABLE sync_outbox
  ADD COLUMN change_type ENUM('INSERT','UPDATE','DELETE','AMENDMENT')
    NOT NULL DEFAULT 'INSERT' AFTER event_type,
  ADD COLUMN before_state_json LONGTEXT NULL AFTER payload_json,
  ADD COLUMN amendment_reason VARCHAR(500) NULL,
  ADD COLUMN amendment_approver VARCHAR(100) NULL,
  ADD COLUMN amendment_date DATETIME NULL,
  ALGORITHM=INSTANT;
```

6.6.5 NLDR-side action by change_type

change_type	NLDR action
INSERT	Create record (idempotent — check event_id first)
UPDATE	Update record (idempotent — version check via payload_hash)
DELETE	Mark is_deleted=1 on NLDR's copy; never physical-delete; retain in amendment_history
AMENDMENT	Update record to after-state; insert amendment_history row with before, after, reason, approver, amendment_date; if amendment_date < first_synced_at, flag as backdated for central audit review

6.6.6 Capture options

Two implementation options exist for capturing the DELETE event:

- **Option A — Application-level capture (recommended):** Modify each delete/amendment workflow in the business services to write the outbox row in the same DbContext transaction. Cleanest, most testable, and integrates naturally with the existing CorrectionTool logging.
- **Option B — MySQL BEFORE DELETE trigger (fallback):** For tables whose DELETES do not flow through the application layer (legacy admin scripts, ad-hoc tooling). If the trigger fails to write the outbox row, the DELETE is aborted; nothing is lost. Triggers are heavier and harder to test, so Option A is preferred for new code paths.

6.6.7 Worst-case recovery scenarios

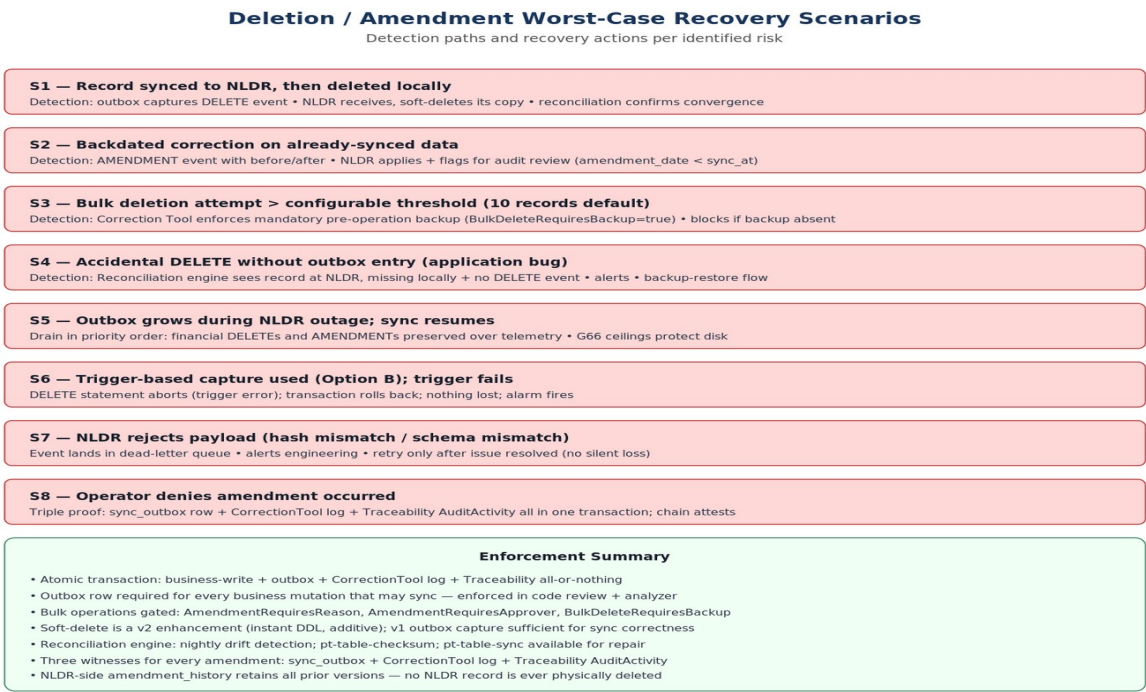


Figure: Deletion / Amendment Worst-Case Recovery Scenarios

6.6.8 Bulk-deletion guardrails

Bulk deletion is the highest-risk operation. The Correction Tool enforces:

- Configurable threshold (default 10 records) above which bulk delete is treated as bulk.
- Mandatory pre-operation backup — bulk delete is blocked unless a verified backup exists within the last hour.
- Each row in the bulk produces its own outbox row (no batched events) — preserves NLDR-side replay simplicity.
- Operator typed confirmation "BULK-DELETE <count>" required before execution.
- Override Token required if backup target is unavailable (governance-gated).

6.6.9 Three witnesses for every amendment

Every amendment writes three records inside the same atomic transaction:

- sync_outbox row with change_type=AMENDMENT and full before/after state.
- CorrectionTool_<module>ActivityLog row with operator and reason.
- erp_traceability.AuditActivity row written via the AuditHook with geo-tag and ULID.

Three independent records, all hash-chained or chain-attested. An operator denying "I did not change this" is contradicted by three witnesses, all immutable in their respective audit systems.

6.6.10 Soft-delete recommendation (v2 enhancement)

For v2, adding `is_deleted`, `deleted_at`, `deleted_by`, `deletion_reason` columns to critical financial tables — using `ALGORITHM=INSTANT` (additive, non-breaking) — would replace hard `DELETE` with soft-delete `UPDATE`. Application queries would add `WHERE is_deleted=0` to all `SELECT`s. This is a v2 enhancement; for v1 the outbox-based event capture is sufficient for sync correctness without requiring schema changes to all 1,057 tables.

6.6.11 Configuration

```
{
  "SyncDeletion": {
    "CaptureDeleteEvents": true,
    "CaptureAmendmentEvents": true,
    "UseTriggers": false,
    "UseApplicationCapture": true,
    "AmendmentRequiresReason": true,
    "AmendmentRequiresApprover": true,
    "BulkDeleteThreshold": 10,
    "BulkDeleteRequiresBackup": true,
    "RetainDeletedStateInOutboxDays": 365
  }
}
```

6.7 AUTO_INCREMENT Seed and BIGINT ID-Space Partitioning

When hundreds of PACS sites generate primary-key values offline and later sync to a single NLDR, naive `AUTO_INCREMENT` values would collide on every key column. ePACS solves this not with UUIDs (which would inflate indexes and cost storage on 1,057 tables × billions of rows) but with deterministic range partitioning of the `BIGINT` ID space. Each PACS receives a disjoint 10-billion-slot range derived from its numeric `pacsid`. The result is offline-safe, infinite-feeling per-PACS ID generation with zero cross-PACS or PACS ↔ NLDR collision risk.

Design principle: "Primary keys belong to one PACS forever." Every `BIGINT` primary key in ePACS encodes its originating site in the high digits. A row with ID 21,150,000,000,142 is unambiguously the 142nd row created at PACS 2115. NLDR cannot generate that ID; PACS 2116 cannot generate that ID; therefore sync is collision-free by construction, with no central coordination required and no UUID overhead.

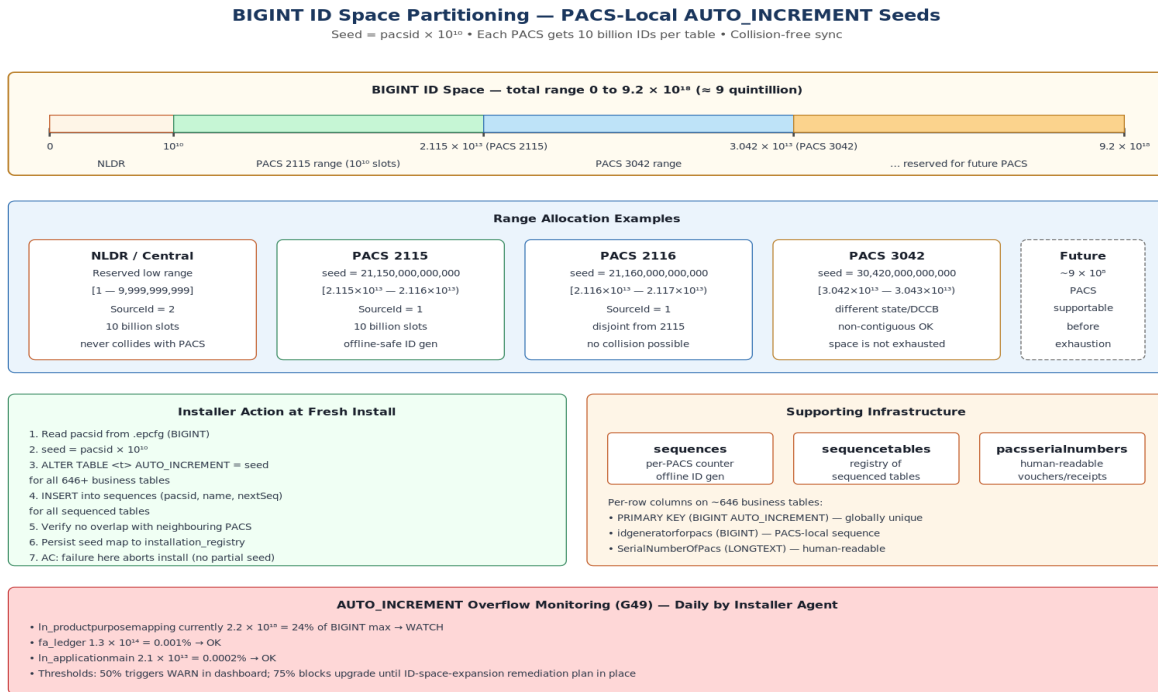


Figure: BIGINT ID-Space Partitioning — Per-PACS Seeds, Sequence Tables, Overflow Monitoring

6.7.1 The seed formula

```
// Single source of truth: pacsid from .epcfg
const long RANGE_SIZE = 10_000_000_000L; // 10^10 = 10 billion slots per PACS per table
long seed = pacsid * RANGE_SIZE;
```

// Examples

// PACS 2115 → seed 21_150_000_000_000 → range [2.115×10¹³ ... 2.116×10¹³)

// PACS 2116 → seed 21_160_000_000_000 → range [2.116×10¹³ ... 2.117×10¹³)

// PACS 3042 → seed 30_420_000_000_000 → range [3.042×10¹³ ... 3.043×10¹³)

// NLDR → reserved low range [1 ... 9_999_999_999] (SourceId = 2)

BIGINT max is 9.223×10^{18} . Dividing by 10^{10} slots-per-PACS yields headroom for $\sim 9 \times 10^8$ PACS sites — orders of magnitude beyond the deployment target — before space-exhaustion becomes a concern.

6.7.2 Per-row provenance columns

About 646 business tables carry four columns that together enable offline ID generation, sync provenance, and human-readable identifiers:

Column	Type	Role
PRIMARY KEY (SIno / ApplicationNo / VoucherNo / ...)	BIGINT AUTO_INCREMENT	Globally unique. Seeded per-PACS at install. Never collides across PACS or with NLDR.
idgeneratorforpacs	BIGINT	PACS-local sequence counter for offline ID generation when AUTO_INCREMENT cannot be used (e.g., composite-key derivation).
SerialNumberOfPacs	LONGTEXT	Human-readable serial (voucher numbers, receipt numbers) — distinct from internal PK; user-facing.

Column	Type	Role
Sourceld	TINYINT	Origin marker. 1 = local PACS create, 2 = received from NLDR, 3 = legacy migration import.

6.7.3 Supporting infrastructure tables

Table	Purpose
sequences	Per-PACS, per-table sequence counters: (pacsid, sequencename, nextsequenceid). Used by application code that increments offline (Hi-Lo style) without round-tripping to MySQL for every ID.
sequencetables	Registry of which tables participate in sequence-based ID generation; read by installer to know which tables to seed.
pacsserialnumbers	PACS-specific human-readable serial number registry. Distinct from internal PKs; tracks voucher/receipt numbering policies (per-financial-year resets, prefix conventions).

6.7.4 Installer responsibilities at fresh install

Seeding is a critical, one-time, all-or-nothing operation at install time. The installer:

- **Step 1:** Read pacsid from the signed .epcfg (BIGINT, e.g., 2115).
- **Step 2:** Compute seed = pacsid × 10¹⁰ once and reuse for every business table.
- **Step 3:** Read /config/sequence-tables.yaml — the inventory of 646+ business tables to seed.
- **Step 4:** For each table, run ALTER TABLE <t> AUTO_INCREMENT = <seed>. Idempotent (G75): re-running install must produce identical seeds.
- **Step 5:** INSERT into sequences for every sequenced table with nextsequenceid = seed and Sourceld = 1.
- **Step 6:** Verify no overlap with neighbouring PACS (sanity check; impossible by construction unless pacsid was reused — installer fails fast).
- **Step 7:** Persist the seed map and pacsid to installation_registry for audit; record event in Traceability.
- **Acceptance:** Failure at any step aborts the install — no partial seeding is permitted, because a partially-seeded database could produce collision-prone IDs after the install completes.

6.7.5 Sync collision prevention

Direction	Behavior
PACS → NLDR	Records created locally have PKs in the PACS's range. NLDR stores them with the same PK (no re-keying). Sourceld = 1 marks them as PACS-originated.
NLDR → PACS	Records pushed by NLDR carry IDs from the reserved low range [1 ... 9,999,999,999]. Sourceld = 2. Cannot collide with any PACS-generated ID.
PACS A → PACS B (via NLDR)	PACS A's ID is preserved end-to-end. PACS B sees a row with Sourceld = 1 but a PK from PACS A's range — clearly not local.
Legacy import	Sourceld = 3. Legacy IDs were assigned at original migration time; the seed approach was applied only to new tables and post-migration rows.

6.7.6 Configuration

```
{
  "IdGeneration": {
```

```
"RangeSize": 10000000000,
"PacsIdSource": "epcfg",
"NldrReservedRange": { "Min": 1, "Max": 9999999999 },
"ValidateNoOverlap": true,
"SequenceTablesPath": "config/sequence-tables.yaml"
}
}
```

6.7.7 AUTO_INCREMENT overflow monitoring (G49)

Although the per-PACS range is effectively infinite for normal business volumes, certain composite-key tables approach the BIGINT ceiling because they encode pacsid into the high digits and burn through the low digits at high transaction rates. The Installer Agent monitors AUTO_INCREMENT values daily.

Table	Current AUTO_INCREMENT	BIGINT max	Usage %	Verdict
In_productpurposemapping	2.2×10^{18}	9.2×10^{18}	24 %	WATCH
fa_ledger	1.3×10^{14}	9.2×10^{18}	0.001 %	OK
In_applicationmain	2.1×10^{13}	9.2×10^{18}	0.0002 %	OK

Thresholds: 50 % usage triggers a WARNING in the health dashboard; 75 % blocks any further upgrade until an ID-space-expansion remediation plan is in place. The remediation options are (a) compaction (rare in this schema), (b) decimal-place reallocation in the seed formula, or (c) migration to a 128-bit ID column for the specific table.

6.7.8 Why not UUID?

UUIDs were considered and rejected for ePACS for three reasons: (1) UUID v4 randomness destroys clustered-index locality on InnoDB, causing severe write amplification on hot tables like fa_ledger. (2) 16-byte UUID indexes double the storage and RAM footprint compared to 8-byte BIGINT — non-trivial on the 4,556-LOB-column schema. (3) UUIDs do not encode provenance — "which PACS created this row?" requires either a separate column or a lookup. The range-partitioned BIGINT scheme delivers all the collision-avoidance benefits of UUIDs while preserving locality, halving index size, and encoding origin in the value itself. UUID v7 (time-ordered) was considered later but adopted only for transient identifiers (correlationId, event_id) where storage cost is negligible.

7. How Online ↔ Offline Operation Works

7.1 Design principle

MySQL on the local PACS node is always the authoritative source of truth for business data. NLDR is a downstream system; sync to NLDR is opportunistic. Connectivity loss to NLDR never blocks business operations. Connectivity restoration triggers idempotent drain of accumulated events.

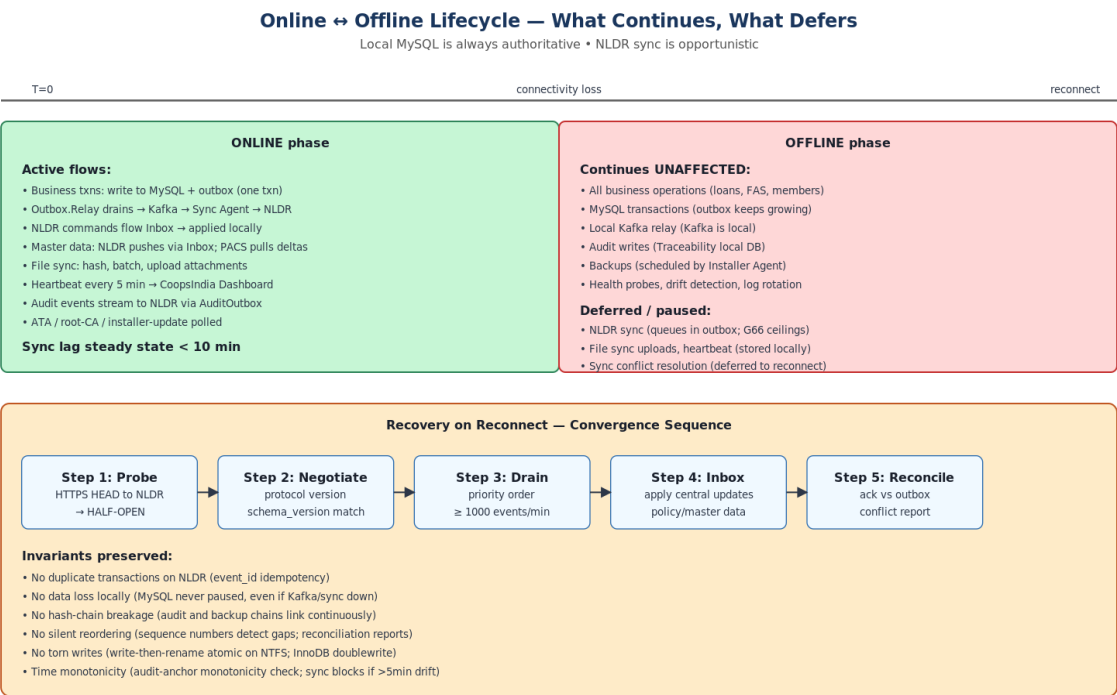


Figure: Online ↔ Offline Lifecycle — What Continues, What Defers, How Convergence Happens

7.2 What works offline

Capability	Behavior offline
Business operations	All loans, deposits, FAS, membership, trade transactions execute against local MySQL with full ACID.
Audit logging	Traceability module writes to local erp_traceability DB; hash chain continues.
Local Kafka relay	Outbox.Relay continues to drain MySQL outbox into local Kafka topics.
Cache	Garnet operates normally; MySQL refills on miss.
Backups	Installer Agent runs scheduled backups against local data.
Health, drift, log rotation	All Installer Agent monitors continue operating.
File sync (local file capture)	FileSyncMonitor continues hashing and queuing files; uploads queue.
Operator ePACS Web	Available on PACS LAN at HTTPS 443.
epacs-ping.exe	Returns local-only health summary; sync state shows OPEN circuit.

7.3 What pauses offline (queues for later)

Capability	Behavior
NLDR sync (outbound)	Events queue in MySQL sync_outbox; circuit moves to OPEN; drain on reconnect.
NLDR commands (inbound)	Cannot arrive while offline; processed in priority order on reconnect.
File sync uploads	Hash and batch persisted; uploads queued; resume from last ACK'd chunk on reconnect.
Heartbeat to CoopsIndia	Fire-and-forget; circuit-breaker silent; resumes on reconnect (no historic backfill).
Fleet Health Manifest	Same as heartbeat; opportunistic upload.
Sync conflict resolution UI	Conflicts visible only after reconnect drains pending events; deferred decisions logged.
ATA / root-CA / installer self-update	Polled when sync window available.

7.4 Connectivity state machine

ConnectivityMonitor implements a circuit breaker with three states. Bandwidth detection on every probe enables adaptive chunking — 1 MB chunks on 4G, 256 KB on 3G, 64 KB on 2G — so file sync remains responsive on very slow links.

7.5 Recovery sequence on reconnect

When the circuit transitions back to CONNECTED, a strict convergence sequence runs:

- **Step 1 — Probe:** HTTPS HEAD to NLDR endpoint; latency measured for bandwidth tier.
- **Step 2 — Negotiate:** Sync protocol version handshake; schema_version match check; ATA refresh.
- **Step 3 — Drain outbox:** Priority order — financial → audit → anomaly flags → master data → file uploads → fleet manifest. Target ≥ 1,000 events/min.
- **Step 4 — Process inbox:** Apply central updates (master data, policies); resolve conflicts via decision tree.
- **Step 5 — Reconcile:** Compare ACKs vs outbox; emit ReconciliationReport; surface anomalies in health dashboard.

7.6 Bounded outbox growth (G66)

Without bounds, a 30+ day outage at NLDR could fill disk and choke MySQL. Backpressure thresholds:

- Soft ceiling 500 K events → health dashboard YELLOW; "days to ceiling" gauge displayed to operator.
- Hard ceiling 2 M events → telemetry/non-financial events deferred to sync_outbox_overflow cold-storage table.
- Financial transactions are NEVER deferred — they remain in main outbox until delivered.
- Audit events maintain priority ordering even under backpressure.

7.7 File sync architecture (offline-aware)

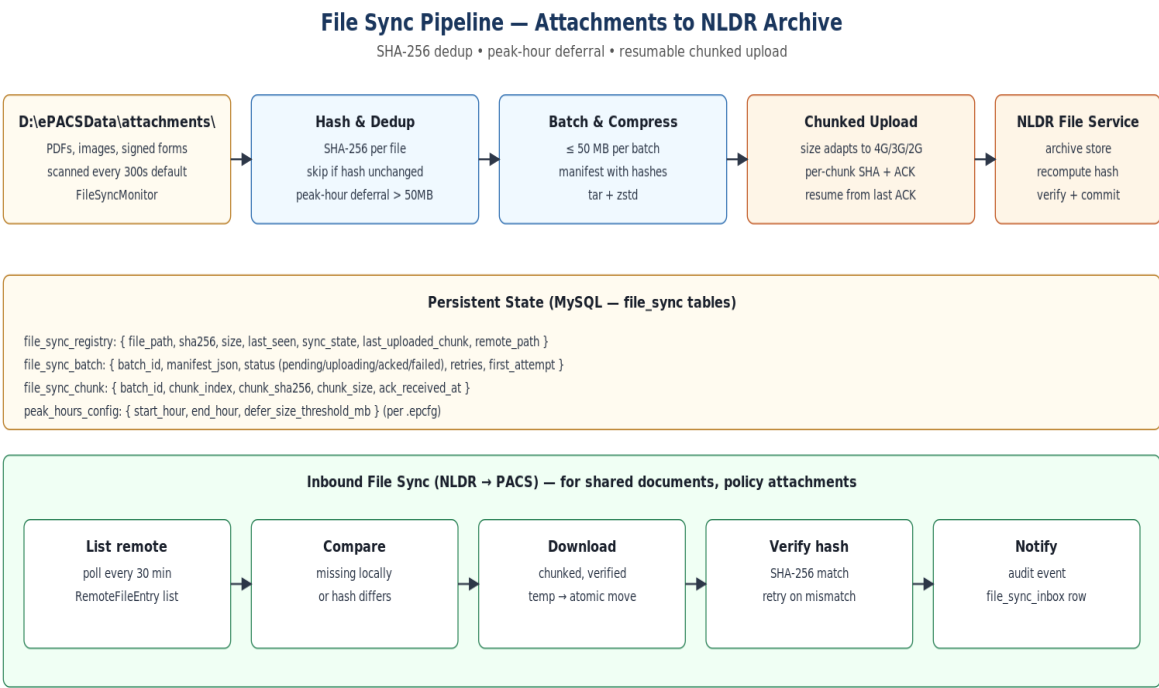


Figure: File Sync Pipeline — Hash Dedup, Peak-Hour Deferral, Resumable Chunked Upload

7.8 30-day offline drill (validation)

The 30-day offline drill is part of Phase 3 exit criteria. It validates that:

- All business operations continue; no service crashes; no data loss.
- Backups complete daily; chain attestation continuous.
- Outbox grows but stays under 2 M ceiling; financial transactions retained.
- On reconnect, drain completes in priority order; ≥ 1,000 events/min.
- No duplicates at NLDR; no out-of-order; no hash mismatch.

8. Patterns, Best Practices, and Scenarios Covered

8.1 Patterns applied

Pattern	Where used
Transactional Outbox	Sync events committed atomically with business writes (Sync.Agent)
Dead Letter Queue	Kafka epacs.local.dead-letter for sync events that exhausted retries
Circuit Breaker	ConnectivityMonitor (Connected/Half-Open/Open) for NLDR
Polly retry + jittered backoff	All outbound HTTP and Kafka publish operations
Side-by-Side Deployment	C:\Program Files\ePACS\releases\<v>\ + junction-flip
Expand-Migrate-Contract	Schema migration phases enforced by DbUp script header
Saga (compensating actions)	Upgrade rollback: stop-flip-restore as compensations of stage-flip-start
Hexagonal / Ports-and-Adapters	ISyncTransport, ICache, ICodeSigner, IFileSyncTransport, IAuditChain
Strategy	DDL classifier picks INSTANT/INPLACE/COPY; pt-osc routing
State Machine	Installer phases with durable checkpoints and recovery
CQRS-light	Read-side health endpoints cached separately from authoritative MySQL writes
Worker / Background Service	Installer.Agent, Sync.Agent, Outbox.Relay as Windows services
Configuration-as-Code	.epcfg signed pack; appsettings hierarchies; option pattern
Hash Chain (Merkle-light)	Audit log; backup manifest chain (G87)
Repository / Unit-of-Work	BackupEngine, RestoreEngine, MigrationRunner own transactions
Factory	IInstallerErrorFactory creates typed exceptions stamped with correlation_id
Observer / Monitor	IMonitor implementations under Installer.Agent
Idempotent Receiver	InboxProcessor checks event_id against sync_inbox before applying
Two-Person Rule	Production signing pipeline (G67)
Kill-Switch / Allow-List	ATA (G68) for code-signing key compromise response
Tombstone / Soft Delete	Critical tables retain deleted_at instead of hard DELETE for audit (NLDR side)
Catalog-driven Errors	YAML error catalog + IErrorFactory; user/support/code messages decoupled
Three Witnesses	Atomic write of sync_outbox + CorrectionTool log + Traceability for non-repudiable audit
Before-State Capture	DELETE / AMENDMENT events carry pre-mutation state for replay and reconciliation
Audit Hook Bridge	Logging events tagged audit-relevant flow into Traceability via AuditHook adapter
Correlation Propagation	ULID correlationId flows HTTP → outbox → Kafka → sync agent → NLDR
Range-Partitioned IDs	BIGINT ID space partitioned per-PACS via seed = pacsid × 10 ¹⁰ ; collision-free sync without UUIDs
Hi-Lo / Sequence Allocator	sequences table provides offline-safe ID generation when AUTO_INCREMENT cannot be

Pattern	Where used
	used
Provenance-Encoded Keys	PK value itself encodes originating PACS in the high digits; SourceId column reinforces provenance

8.2 Best practices encoded

- All file writes are write-then-rename atomic; no torn writes possible.
- Service registration is idempotent — running install twice yields byte-identical state (Appendix L of plan).
- Locale-sensitive parsers banned (DateTime.Parse, decimal.Parse without InvariantCulture); analyzer enforces (G73).
- All datetimes serialized as ISO 8601 UTC; localized only at UI.
- Path comparisons use OrdinalIgnoreCase; UTF-8 file I/O explicit.
- File names normalized to NFC at upload (Indic script duplicate prevention).
- Process-level mutex + MySQL advisory lock (belt-and-suspenders) prevent concurrent runners.
- Backup target validation rejects FAT32/SMB for physical backup; only logical can target removable media.
- Health endpoints rate-limited and cached (G79) — no MySQL hammering by external probes.
- All option classes (BackupOptions, FileSyncOptions, etc.) validated on app start; missing/invalid values fail fast with clear errors.
- All long operations write progress checkpoints; can resume after interruption.
- Critical operations require Override Token + typed confirmation (purge, force-upgrade, accept-conflict-local).

8.3 Anti-patterns explicitly avoided

- No dual-write across MySQL and Kafka — outbox pattern instead.
- No use of Kafka offset alone for sync checkpointing — durable in MySQL sync_checkpoints.
- No browser localStorage for state — all state in MySQL or signed durable files.
- No mutable state in side-by-side releases directory — releases\<v>\ is immutable post-install.
- No auto-format of existing Kafka log.dirs — format only on empty directory.
- No INSERT IGNORE used to mask FK violations (only used intentionally for idempotent seeds).
- No password equality comparison via String.Equals — fixed-time compare used.
- No SELECT * in migration scripts — explicit columns to avoid drift surprises.

8.4 Scenarios covered (with mitigation)

Scenario	Behavior / mitigation
Power-cut mid-install (extraction phase)	state.json checkpoint; on restart, RECOVERY mode resumes from last extracted payload.
Power-cut mid-upgrade (migration phase)	DbUp checkpoint per script; advisory lock cleared; runner resumes from last committed script.

Scenario	Behavior / mitigation
Power-cut mid-junction-flip	Junction is atomic; either old or new active; no in-between.
Power-cut mid-backup	Manifest only signed at end; incomplete backup discarded; previous valid backup retained.
Power-cut mid-restore	Pre-restore safety backup exists; revert and retry.
MySQL crash	innodb_doublewrite + crash recovery; service auto-restart; agent collects support bundle on 5 fails.
Garnet AOF corruption	Quarantine AOF; start empty cache; refill from MySQL; 6h snapshots in backup (G91).
Kafka unclean shutdown	flush.messages=1 + flush.ms=1000; KRaft single-node metadata recovery; never auto-format existing datadir.
AV quarantines binary	Detection in agent; restore from media or quarantine; Authenticode + ATA re-verified.
Disk full during backup	Backup target validation pre-check (1.5x size); adaptive eviction (G84) at < 20% free.
Disk full during business ops	Tiered alerts (G24): Yellow 20%, Red 10%, Critical 5%; Critical blocks non-essential writes.
Clock drift > 5 min	Sync blocked; agent warns; operator runs w32time sync; audit-anchor monotonicity check.
NLDR offline 30+ days	Outbox grows under ceilings (G66); financial preserved; drain on reconnect.
USB media corrupted	SHA-256 verify before extraction (G33); per-part hashes for split media; clear E019 error.
Concurrent installer execution	Process mutex (G32) + MySQL advisory lock (G76); E018 / E021.
Schema drift (manual DDL)	Fingerprint diff (G45); breaking drift blocks upgrade; remediation script generated.
Skip-version upgrade (N-2 → N)	DbUp detects gap; applies cumulative deltas; tested per supported skip path (G90).
Restore on different hardware class	Cross-profile detection (G85); regenerate my.ini, Garnet limits, Kestrel pool.
Code-signing key compromised	Compromise runbook (Appendix M); ATA blacklist deployed within 24h; new cert + re-sign.
GPO disables ePACS service	GPO compatibility precheck (G72); explicit service-disabled-by-policy alert (not drift).
Stale Windows root-CA	Bundled root-CA snapshot per release; staleness health check at 18 / 24 months (G71).
NTP unavailable (air-gapped)	Multi-tier fallback (G70); audit-anchor monotonicity; weekly operator time check.
Operator runs install twice	Idempotency invariants (G75) — byte-identical state after second run.
Operator force-kills installer	RECOVERY mode on next launch resumes from last checkpoint.
Operator manually edits config	Drift detection (G22) flags; repair mode restores from templates if requested.
Operator types wrong PACS ID	.epcfg-signed PACS ID prevails; manual entry validated against state registry format.
Override Token replay	Nonce store rejects (G14).

Scenario	Behavior / mitigation
Sync forgery attempt	mTLS + payload SHA-256 + sequence + nonce; rejected at multiple layers.
Indic script roundtrip (Hindi name with conjuncts)	utf8mb4_0900_ai_ci validated; NFC normalization; locale test corpus (G73).
DST transition during scheduled task	Time-skip tests cover transitions (G74); scheduler uses UTC internally.
Large dataset migration (100 GB DB)	XtraBackup incremental + pt-online-schema-change for COPY DDL on > 1M-row tables.
AUTO_INCREMENT overflow risk	Daily monitoring (G49); 50 % triggers warning; 75 % blocks upgrade.
File sync floods 4G uplink	MaxUploadBatchSizeMb cap; bandwidth detection adjusts chunk size; small files first (G93).
SFTP key management at scale	Per-PACS keypair generated at install; pubkey registered with NLDR via sync; rotation via signed config (G94).
Heartbeat endpoint down	Fire-and-forget; circuit breaker; failure logged but never blocks business (G95).
Hash compute on large PDFs slows host	Background thread, low priority; max 1 file at a time; > threshold deferred during peak hours (G96).
Record synced to NLDR then deleted locally	Outbox captures DELETE event with before-state; NLDR soft-deletes its copy; reconciliation confirms (G97).
In-place amendment on synced data	AMENDMENT event with before+after state; NLDR applies + flags backdated changes for audit review (G98).
Bulk deletion (> 10 records) without backup	Correction Tool blocks; mandatory verified backup within 1 hour; typed confirmation; per-row outbox events (G99).
Operator denies amendment occurred	Three witnesses inside one transaction: sync_outbox + CorrectionTool log + Traceability AuditActivity.
Application bug deletes without outbox event	Reconciliation detects: row gone locally, present at NLDR, no DELETE event → operator alert + investigation.
PACS PKs collide across sites at NLDR	Impossible by construction: each PACS owns a disjoint 10^{10} -slot BIGINT range derived from pacsid (Section 6.7).
Two PACS accidentally given same pacsid	Installer fails fast: state federation portal validates pacsid uniqueness when issuing .epcfg; runtime overlap check at install.
Restore on a different PACS site (wrong pacsid)	.epcfg pacs_id mismatch → restore blocked (E016); recovery requires correct .epcfg or governance Override Token.
Partial seed (installer crashes mid-ALTER on table 200/646)	All-or-nothing: install transaction aborts; state.json checkpoint causes RECOVERY mode to retry; idempotency guarantees byte-identical seeds on retry (G75).
AUTO_INCREMENT approaches BIGINT ceiling on a hot table	Daily monitor (G49): 50 % triggers WARNING; 75 % blocks upgrade; remediation = ID-space-expansion plan or table-specific 128-bit migration.
NLDR pushes record back to PACS with NLDR-range PK	Sourcelid = 2 marks origin; cannot collide with any PACS-generated PK; idempotent apply via event_id.

9. Glossary

Term	Definition
ACL	Access Control List — Windows file-system permission set.
ATA	Anti-Tamper Allowlist — signed list of trusted code-signing certificate thumbprints, refreshed via sync. Supports rapid revocation if a signing key is compromised.
AC-001 ... AC-012	Acceptance criteria from BRD.
Authenticode	Microsoft technology for digitally signing executable files.
Backup chain	Merkle-style linkage where each backup manifest references the previous backup's signature, enabling tamper detection.
BRD	Business Requirements Document.
BR sigil naming	ePACSWeb, ePACSMySQL, ePACSCache, ePACSEventing, ePACS-Loans, ePACSSync, ePACSInstallerAgent — Windows service names.
Burn	WiX v4 bundle/chain installer technology. Hosts a Managed Bootstrapper Application written in C#.
Circuit breaker	Pattern with Connected/Half-Open/Open states to fail fast on persistent downstream failures.
CoopsIndia Dashboard	Central HQ dashboard for fleet visibility (heartbeats and Fleet Health Manifest).
Correlation ID	UUID v7 stamped on every operation; propagated to logs, audit, sync envelope, support bundle.
DCCB	District Central Cooperative Bank — larger PACS hub site.
DbUp	.NET-native forward-only schema migration framework with checkpoint-per-script.
DPAPI	Windows Data Protection API — machine-bound key derivation.
Drift (config)	When generated configuration files differ from the hash recorded at install/upgrade time.
Drift (schema)	When the live MySQL schema differs from the expected baseline for the installed version.
EV cert	Extended Validation Authenticode certificate; trusted by Windows SmartScreen.
Expand-Migrate-Contract	Three-phase schema change pattern that preserves rollback capability across releases.
.epcfg	Site Configuration Pack — signed and encrypted JSON descriptor for a specific PACS installation.
Fingerprint (schema)	INFORMATION_SCHEMA-derived snapshot of tables, columns, indexes, FKs, charset, collation, hashed for drift comparison.
Fleet Health Manifest	Privacy-minimized telemetry payload from PACS to CoopsIndia (versions, health, sync lag, disk %, cert expiry).
Garnet	Microsoft OSS Redis-compatible cache built on .NET 8.
Gap (G##)	Risk or missing capability identified in the plan, with a documented mitigation.
Heartbeat	Periodic liveness ping from PACS to CoopsIndia Dashboard (HTTPS or WebSocket).
HSM	Hardware Security Module.
Idempotency key	Application-level identifier to ensure replays do not produce duplicate effects.

Term	Definition
in-toto	Framework for collecting and verifying provenance attestations across software supply chain.
Junction	NTFS reparse point used for atomic version switching (current → releases\<v>).
Kestrel	Cross-platform .NET web server hosted under Windows Service.
KRaft	Kafka's Zookeeper-less consensus mode (single-node deployment supported).
LSN	Log Sequence Number — InnoDB redo log marker, used by XtraBackup for incremental delta.
Manifest (release)	Signed YAML describing all payloads, hashes, sizes, install order, compatibility.
mTLS	Mutual TLS — both client and server present certificates.
NLDR	National Level Data Repository — central destination for cooperative-banking data.
NLPSV	National-Level PACS Software Vendor — issues releases, signing, governance.
NFC	Unicode Normalization Form C — canonical composition; required for filename consistency.
Outbox	MySQL table holding events to be published; drained transactionally with business writes.
Override Token	Signed JWT (issued by Release CA) authorizing a specific destructive action with TTL and nonce.
pt-osc	pt-online-schema-change — Percona tool for non-blocking DDL on large tables via shadow copy.
pt-table-checksum	Percona tool for chunk-based row-checksum comparison across MySQL instances.
pt-table-sync	Percona tool for repairing data drift detected by pt-table-checksum.
PACS	Primary Agricultural Credit Society — local cooperative bank.
Reconciliation	Nightly comparison of local outbox checkpoints against NLDR ACKs to detect drift.
Release CA	Certificate authority operated by NLPSV; signs releases, manifests, ATA, recovery certs.
RFC 3161	Time-stamping protocol for code signatures, ensuring signatures remain valid past cert expiry.
SAD	Solution Architecture & Design — this document.
SBOM	Software Bill of Materials — CycloneDX JSON listing all components and hashes.
SLSA	Supply-chain Levels for Software Artifacts — graduated maturity model for build-pipeline integrity.
Smoke test	Post-install API test (create, verify, delete a record + audit verification) confirming end-to-end health.
State.json	Durable installer checkpoint file at D:\ePACSDData\installer\state.json.
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of Service, Elevation of privilege — threat-modeling taxonomy.
Stryker.NET	.NET mutation testing framework.
Support bundle	Diagnostic archive collected for engineering escalation; secret-redacted; signed.
TPM	Trusted Platform Module — hardware-backed key storage available on most modern

Term	Definition
	Windows machines.
Traceability module	Compliance-grade audit subsystem (erp_traceability database, 11 partitioned tables, ULID PKs, geo-tagged, anomaly rules).
Trust boundary	Architectural seam at which authentication or validation must occur (TB-1 through TB-7).
XtraBackup	Percona OSS physical backup tool — InnoDB page-level incremental, no table locks.
WiX	Windows Installer XML toolset; v4 used.
AMENDMENT (event)	Sync event class indicating an in-place UPDATE for a record that may have already synced; carries before-state, after-state, reason, approver, amendment_date.
AppException	Base class of the typed exception hierarchy in Intellect.Erp.ErrorHandling; carries error code, correlationId, severity, retryable hint.
AppLogger<T>	Serilog-backed logger from Intellect.Erp.Observability.Core; the only logger used in ePACS code.
AuditHook	Bridge from Observability framework to Traceability module; production default at PACS is TraceabilityBridge.
before_state_json	Pre-mutation state captured atomically with DELETE/AMENDMENT and propagated to NLDR for reconstruction and audit.
change_type	Discriminator on sync_outbox indicating INSERT / UPDATE / DELETE / AMENDMENT.
CorrectionTool	ePACS administrative tool that logs corrections to Loans / Membership; logs in CorrectionTool_<module>ActivityLog tables.
CorrelationDelegatingHandler	HttpClient handler from Intellect.Erp.Observability.Propagation that adds X-Correlation-Id header to outbound HTTP calls.
Deletion-pattern tables	Six tables that capture pre-deletion state across modules: fa_voucherdeletionmain/details, pds_purchaseddelete/salesdelete, tr_purchaseddelete/salesdelete.
Error catalog (YAML)	Per-module YAML file listing every error code with userMessage, supportMessage, severity, retryable, category.
IErrorFactory	Factory in Intellect.Erp.ErrorHandling that constructs AppException from a catalog code; stamps correlationId.
IRedactionEngine	Service that strips PII (Aadhaar, mobile, account, email) from log entries and support bundles based on attribute decoration.
KafkaHeaders	Helper from Intellect.Erp.Observability.Propagation that copies correlationId onto Kafka message headers.
Soft-delete (NLDR)	NLDR-side response to DELETE events: records are marked is_deleted=1 but never physically removed; preserved for audit.
TraceableBackgroundService	Base class for worker services that auto-generates correlationId per cycle, pushes context to Serilog, catches and logs exceptions.
Three witnesses	Every amendment writes three independent records inside one transaction: sync_outbox + CorrectionTool log + Traceability AuditActivity.
AUTO_INCREMENT seed	BIGINT value computed as pacsid $\times 10^{10}$; applied via ALTER TABLE on every business table at fresh install; ensures collision-free PK generation across PACS sites.
BIGINT ID space	The 8-byte signed integer range $[0 \dots 9.223 \times 10^{18}]$ used for primary keys across ePACS;

Term	Definition
	partitioned per-PACS in 10-billion-slot chunks.
idgeneratorforpacs	BIGINT column on ~646 business tables; PACS-local sequence counter for offline ID generation when AUTO_INCREMENT cannot be used directly.
NLDR reserved range	ID range [1 ... 9,999,999,999] reserved for NLDR-originated records; never assigned to any PACS.
pacsid	Numeric BIGINT site identifier supplied via signed .epcfg; the high-digit prefix of every PK created at that PACS; immutable for the lifetime of the installation.
pacsserialnumbers	Registry table of human-readable serial number policies (voucher / receipt prefixes, financial-year resets) per PACS.
RANGE_SIZE	10^{10} (10 billion). The number of ID slots allocated per PACS per table; ensures no PACS will ever exhaust its range under normal business volumes.
seed formula	$\text{seed} = \text{pacsid} \times \text{RANGE_SIZE} = \text{pacsid} \times 10^{10}$. The single derivation rule used by the installer to compute every per-PACS AUTO_INCREMENT seed.
sequences (table)	Per-PACS, per-table sequence counter table — { pacsid, sequencename, nextsequenceid, SourceId } — used by application code that increments offline.
sequencetables (table)	Registry of which business tables participate in sequence-based ID generation; read by installer to drive seeding.
SourceId	TINYINT column on business tables identifying record origin: 1 = local PACS create, 2 = received from NLDR, 3 = legacy migration.

10. References

10.1 Internal documents

- PACS_ERP_Offline_Installer_BRD — Business Requirements Document (NLPSV).
- ePACS_Offline_Installer_Plan v3.2 — Critical Analysis and Enhanced Implementation Plan (92 gaps, 20 sections, 14 appendices).
- ePACS_Full_Enterprise_Installer_BRD_and_Technical_Architecture — extended technical architecture.
- Updated BRD and deep-research conclusions — discovery and validation.
- AP_DDL.sql — schema reference (1,057 tables, 81 FKs, 4,556 LOB columns, mixed charsets).
- ADRs 0001 – 0020 under /docs/adr/.

10.2 Standards and frameworks

- SLSA — Supply-chain Levels for Software Artifacts (<https://slsa.dev/>).
- in-toto — supply-chain attestation framework (<https://in-toto.io/>).
- CycloneDX — SBOM specification (<https://cyclonedx.org/>).
- STRIDE threat-model methodology (Microsoft SDL).
- RFC 8785 — JSON Canonicalization Scheme (JCS).
- RFC 3161 — Internet X.509 Public Key Infrastructure Time-Stamp Protocol.

10.3 Technologies and tools

- .NET 8 LTS — <https://dotnet.microsoft.com/>.
- MySQL 8.4 LTS — <https://dev.mysql.com/>.
- Microsoft Garnet — <https://github.com/microsoft/garnet>.
- Apache Kafka 3.7 LTS — <https://kafka.apache.org/>.
- Eclipse Temurin JRE 17 LTS — <https://adoptium.net/>.
- WiX Toolset v4 — <https://wixtoolset.org/>.
- DbUp — <https://dbup.readthedocs.io/>.
- Percona XtraBackup 8.4 — <https://www.percona.com/>.
- Percona Toolkit (pt-online-schema-change, pt-table-checksum, pt-table-sync) — <https://www.percona.com/>.
- Polly — .NET resilience library.
- Stryker.NET — .NET mutation testing.
- SharpFuzz — .NET fuzz testing.
- Pester — PowerShell testing for integration suite.

10.4 Plan references (cross-walk)

This document corresponds to the v3.2 plan. Key plan sections referenced:

This SAD	Plan section / appendix
§4.2 State machine	§3.4 Installer state machine; Appendix B service recovery
§4.5 Side-by-side upgrade	§4 Phase 2; §13.1 power-cut resilience
§4.6 Schema migration	§12 DDL Drift and Differential Migration; Appendix E DDL classification; Appendix F fingerprint
§4.7 Backup and restore	§12.4 Data Delta; Appendix G Percona packaging
§4.8 Sync architecture	§3.5 Sync data contracts; §13.2 connectivity handling
§4.9 SLSA & ATA	§15 Build Provenance & Supply Chain Security; Appendix M compromise runbook
§5 .NET solution	§3.3 repository layout
§5.6.5 STRIDE	§16 Threat Model; Appendix I detailed threat matrix
§6 Data consistency	§13.1 power-cut matrix; §6 risks; §12.5 sync delta
§7 Online ↔ Offline	§13.2 connectivity state machine; G66 outbox backpressure
§8 Patterns / scenarios	§5.2 testing strategy; §13 rural resilience; Appendices J/L/N
§6.6 Deletion / amendment sync	§23 Deletion, Amendment, and Financial-Data Sync Architecture
§6.7 AUTO_INCREMENT seed & ID space	§24 AUTO_INCREMENT Seed and ID Space Partitioning

11. Appendices

11.1 Appendix A — Service start order and recovery

Services start in dependency order; stop in reverse order.

Order	Service	Tech	Recovery (1st / 2nd / subsequent)
10	ePACSMySQL	MySQL 8.4	60s / 120s / 300s + support bundle
20	ePACSCache	Garnet	30s / 60s / 120s + support bundle
30	ePACSEventing	Kafka 3.7	60s / 120s / 300s + support bundle
40	ePACS-Loans, Fas, Mbr, ...	ASP.NET Core 8	30s / 60s / 120s + support bundle
50	ePACSWeb	Kestrel	30s / 60s / 120s + support bundle
60	ePACSSync	Sync.Agent	60s / 120s / 300s + support bundle
70	ePACSInstallerAgent	.NET Worker	10s / 30s / 60s (fastest watchdog)

11.2 Appendix B — Key error codes

Code	Category	Cause / message
E001	OS	Windows version < 10.0.17763 — install blocked
E007–E009	Port	3306 / 6379 / 9092 in use by non-ePACS process — install blocked
E010	Admin	Not running as administrator — install blocked
E015	Config	.epcfg signature invalid — install blocked
E018	Lock	Another installer instance running (process mutex) — block
E019	Media	Payload archive checksum mismatch — install media corrupted
E020	Upgrade	MySQL Upgrade Checker found critical issues — block
E021	Migration	Concurrent migration runner detected (advisory lock) — block
E022	SBOM	Install-time SBOM cross-check failed — install media tampered
E023	ATA	Signing-cert thumbprint not in Anti-Tamper Allowlist — block
E024	Schema	Breaking schema drift detected before upgrade — block
E025	Backup	Pre-upgrade backup verification failed — block
E028	Time	Clock drift > 5 min; sync blocked
E029	XtraBackup	Backup target on incompatible filesystem (FAT32/SMB) — block for physical
E031	Config	.epcfg nonce already consumed (replay) — block
E040–E044	GPO	Group policy conflicts — block or warn per policy

11.3 Appendix C — Performance targets

Operation	Small PACS (8 GB)	DCCB Hub (32 GB)
Fresh install	< 15 min	< 12 min
Patch upgrade (binaries only)	< 10 min	< 8 min
Minor upgrade + 10 GB DB	< 30 min	< 20 min
Major upgrade + 10 GB DB	< 45 min	< 30 min
Major upgrade + 100 GB DB	< 4 hours	< 2 hours
Backup 10 GB DB	< 10 min	< 5 min
Backup 100 GB DB	< 90 min	< 30 min
XtraBackup incremental (5 GB delta)	< 5 min	< 2 min
Restore 10 GB DB	< 15 min	< 10 min
Restore 100 GB DB	< 4 hours	< 1.5 hours
Hotfix apply	< 5 min	< 5 min
Sync drain (10K events)	≥ 1,000/min	≥ 3,000/min
Health probe latency P95	< 500 ms	< 250 ms
epacs-ping diagnostic	< 30 sec	< 15 sec
Support bundle generation	< 2 min	< 90 sec

11.4 Appendix D — Repository layout

Selected projects (full layout in plan §3.3):

```

/src
/Installer.BootstrapperApp # WiX Burn Managed BA + WPF UI
/Installer.Core # state machine, schema fingerprint, upgrade engine
/Installer.Core.SchemaMigration# DbUp runner, DDL classifier, pt-osc wrapper
/Installer.Actions # prechecks, install, service orchestration
/Installer.Actions.Uninstall # uninstall with Override Token
/Installer.Actions.Repair # repair mode
/Installer.Actions.Hotfix # emergency hotfix
/Installer.Actions.SelfUpdate # G64 installer-only patches
/Installer.UI # WPF i18n
/Installer.CLI # silent mode
/Installer.Agent # ePACSInstallerAgent service
/Installer.Agent.* (Monitors, FileSync, Heartbeat, Scheduler, FleetManifest, AOFGuard)
/Installer.Diagnostics.Ping # epacs-ping.exe
/Installer.Security.* (ATA, RootCA, ServiceAuth, SBOMCheck, Locale, GPOCheck, Time)
/Installer.Provenance # in-toto verifier
/BackupRestore # backup/restore + XtraBackup + Chain + Retention + CrossProfile
/ManifestVerifier # parser, hash, signature, SBOM, ATA
/SupportBundle # collector with redaction
/Sync.Abstractions # ISyncTransport, SyncEvent, Ack
/Sync.Agent # outbox, inbox, connectivity, reconciliation
/Sync.Agent.Backpressure # outbox ceilings + overflow
/Sync.Agent.ConflictUI # operator conflict-resolution backend
/Sync.Transport.Http # NLDR HTTP transport

```

```

/Outbox.Relay          # MySQL → Kafka relay
/SharedKernel          # contracts, configuration, security, errors
/services
/ePacs.Web             # Kestrel host + dashboard
/ePacs.Loans /ePacs.Fas /ePacs.Membership /...
/migrations
/V###*.sql            # versioned migrations with classification headers
/R_*.sql              # repeatable (views, procs)
/pre-checks/          # MySQL Upgrade Checker scripts
/charset-remediation/  # utf8mb3→utf8mb4 conversion
/schema-baselines/    # expected fingerprints per version
/tests
/Installer.UnitTests, IntegrationTests, ChaosTests, MutationTests, FuzzTests,
IdempotencyTests, ConcurrencyTests, LocaleTests, TimeSkipTests, GPOTests,
SkipVersionTests, CrossProfileTests, SBOMTests, ReproducibleBuildTests
/Sync.ContractTests, IntegrationTests, BackpressureTests
/BackupRestore.Tests, ChainTests
/SmokeTests, TamperTests

```

11.5 Appendix E — Acceptance criteria coverage

AC	Description	Where covered
AC-001	Clean offline install	§3.2 INSTALL mode; §4.2 state machine
AC-002	Refuse unsigned/tampered	§4.9 SLSA + ATA; §5.6.2
AC-003	Data root + correct ACLs	§3.3 IAclEngine; SharedKernel.Security
AC-004	Health endpoints report versions	§5.5 observability
AC-005	Verified backup before upgrade	§4.5; §4.7 backup chain
AC-006	Upgrade failure → rollback + bundle	§4.5 rollback paths; §4.2 state machine fail branch
AC-007	Uninstall preserves data by default	§3.2 UNINSTALL mode; Override Token for purge
AC-008	Restore from encrypted backup	§4.7; §6.4 tamper-evidence
AC-009	Local txns while NLDR unreachable	§7.2 what works offline
AC-010	Sync without duplicates after reconnect	§6.3.1 idempotency; §6.7 ID-space partitioning; §7.5 recovery sequence
AC-011	Localhost-only DB/cache/eventing	§3.1 service table; §5.6.4 firewall
AC-012	Support bundle: no plaintext secrets	§5.5 bundle redaction; secret-scan CI gate

11.6 Appendix F — Locked decisions summary (with rationale)

Decision	Rationale
WiX v4 + C# Managed BA	OSS, .NET-aligned, mature chain/rollback model, no per-seat cost
Microsoft Garnet (with ICache fallback)	OSS, .NET 8 native, Windows-first; abstraction allows Redis fallback
Kafka KRaft, no ZooKeeper	LTS stability; KRaft eliminates ZK dependency; Temurin OSS
Self-contained .NET 8 publish	Eliminates runtime drift across field sites

Decision	Rationale
Kestrel under Windows Service	Zero IIS dependency; HTTPS via Windows cert store
MySQL 8.4 LTS Community (ZIP)	Avoids MSI conflicts; installer owns full config lifecycle
DbUp + pt-online-schema-change	.NET native + non-blocking COPY-algorithm DDL on large tables
AES-256-GCM with cert-wrapped key	Per-backup unique key; restore-to-new-machine support
Structured JSON daily-rolling logs	Machine-parseable; correlation_id; aggregation-ready
Silent install via .epcfg	Centrally managed rollouts per BRD FR-06
Percona XtraBackup 8.4	Only OSS option for physical incremental; built-in encryption
pt-table-checksum + pt-table-sync	Chunk-based row drift detection and repair
SLSA L3 + reproducible builds	Defense against build-pipeline compromise (XZ/SolarWinds class)
Per-service auth + local mTLS	Defense in depth beyond localhost-only
NTP fallback hierarchy	Air-gapped sites still get monotonic time and tamper-evident audit
Bundled root-CA snapshot	Offline machines stay current without Windows Update
Installer self-update (N→N+1)	Critical bug fixes propagate without site visit
ATA distribution via sync	Rapid revocation if signing key compromised
Outbox ceilings (G66)	Bounded growth under extended NLDR outage
Locale baseline (NFC + utf8mb4_0900_ai_ci + ISO 8601 UTC)	Indic script integrity end-to-end

11.7 Appendix G — Implementation status (v1.0)

Component	Status	Notes
Installer.CLI	v1.0	Silent mode + .epcfg consumption; exit codes mapped
Installer.Core (state machine)	v1.0	Durable checkpoints; recovery on restart
Installer.Core.SchemaMigration	v1.0	DbUp runner + fingerprinter + DDL classifier; pt-osc wrapper
Installer.Actions (prechecks, install)	v1.0	OS, disk, RAM, port, admin, pending reboot, ACL, service orchestration
Installer.Actions.Uninstall	v1.0	Data-preserve default; Override Token gating
Installer.Agent (monitors)	v1.0	Disk, drift, log rotation; smoke test
Installer.Agent.FileSync	v1.0	FileSyncMonitor + IFileSyncTransport; hash dedup; peak-hour deferral
Installer.Agent.Heartbeat	v1.0	HeartbeatMonitor + payload; circuit breaker; HTTPS/WebSocket
Sync.Abstactions / Sync.Agent	v1.0	ISyncTransport; OutboxRelay; InboxProcessor; ConnectivityMonitor; ReconciliationEngine
Outbox.Relay	v1.0	SELECT FOR UPDATE SKIP LOCKED; graceful Kafka-down

Component	Status	Notes
BackupRestore	v1.0	Tier 1–4 backups; restore engine; manifest signing; XtraBackup integration
ManifestVerifier	v1.0	Parser + hash + signature; ATA path
SupportBundle	v1.0	Collector with redaction
SharedKernel	v1.0	Contracts, options, security, errors
Tests (UnitTests, IntegrationTests)	v1.0	Core unit suite + Pester integration scaffolding
Observability framework integration	v1.0	Intellect.Erp.Observability.Core + ErrorHandling + Propagation referenced by Installer.Agent and Sync.Agent; AuditHook=TraceabilityBridge
Error catalog: installer.yaml	v1.0	ERP-INST-PRE/INS/MIG/BAK/SYN/HLT-#### codes mapped from E001–E599
PII redaction engine	v1.0	IRedactionEngine integrated into SupportBundleCollector
Sync — DELETE event capture	v1.0	Application-level capture (Option A); sync_outbox.change_type=DELETE; before_state_json populated from existing deletion-pattern tables
Sync — AMENDMENT event capture	v1.0	Application-level capture in CorrectionTool path; before+after state, reason, approver, amendment_date
Bulk-delete guardrails	v1.0	BulkDeleteThreshold=10; BulkDeleteRequiresBackup=true; typed confirmation; per-row outbox events
Three-witness invariant	v1.0	Atomic transaction writes sync_outbox + CorrectionTool log + Traceability AuditActivity
AUTO_INCREMENT seed & ID-space partitioning	v1.0	Installer reads pacsid from .epcfg; computes seed = pacsid × 10 ¹⁰ ; ALTERs all 646 business tables; seeds sequences and sequencetables; persists to installation_registry
sequence-tables.yaml inventory	v1.0	Configurable list of business tables to seed; idempotent re-application; backed by YAML schema
AUTO_INCREMENT overflow monitor (G49)	v1.0	Installer Agent daily check; 50 % WARN, 75 % BLOCK upgrade; In_productpurposemapping at 24 % flagged for watch
No-overlap validation	v1.0	Pre-install sanity check rejects pacsid that would overlap with neighbouring PACS in installation registry

End of Solution Architecture & Design — v1.0